

Vietnam National University Ho Chi Minh City
Ho Chi Minh City University of Technology
Department of Telecommunications Engineering

—o0o—



SUPERVISOR : Dr.NGUYEN Van Hieu

SUBJECT : Microwave Engineering

PROJECT : Project 4: Sensor node BL and Directional coupler

CLASS : L01

GROUND : 04

Thành viên của nhóm

STT	MSSV	Họ và Tên	Nhiệm vụ	Tiến độ
1	2210214	Lâm Gia Bảo	Viết chương trình Firmware cho Node	100%
2	2210278	Trần Quốc Bảo	Thực hiện thi công phần cứng	100%
3	2210342	Hà Xuân Cát	Thực hiện kiểm định Layout cho Node	100%
4	2210631	Lý Đoàn Dự	Thiết kế Schematic và Layout cho Node	100%
5	2210725	Võ Phát Đạt	Thực hiện thi công phần cứng	100%
6	2210779	Trương Nguyễn Nhật Đông	Thực hiện thi công phần cứng	100%
7	2210780	Nguyễn Đại Đồng	Viết chương trình Firmware cho Node	100%
8	2210838	Đỗ Tiến Giáp	Thiết kế Directional Coupler	100%

Mục lục

1	Giới thiệu tổng quan	1
1.1	Nhiệm vụ của nhóm	1
1.2	Cấu trúc báo cáo đề tài	1
2	Cơ sở lý thuyết	2
2.1	Tổng quan về công nghệ Bluetooth Low Energy (BLE)	2
2.1.1	BLE Beacons	3
2.1.2	BLE protocol stack	3
2.1.3	BLE working modes	6
2.2	Kiến trúc và đặc điểm kỹ thuật của ESP32-C3	6
2.3	Quản lý năng lượng trong BLE node	8
2.4	Lý thuyết về Directional Coupler	9
2.4.1	Nguyên lý vi dải và đường truyền song song (microstrip coupled-line)	9
2.4.2	Tham số tán xạ (S-parameters)	11
2.4.3	Các hệ số đặc trưng	11
2.4.4	Định nghĩa thực dụng	12
2.4.5	Đặc tính thiết kế	13
2.4.6	Bảng thông và đa đoạn	13
3	Thiết kế và thực hiện	14
3.1	Thiết kế node BLE từ chip EPS32-C3	14
3.1.1	Sensor	14
3.1.2	BLE	17
3.1.3	Sơ đồ mạch của một Sensor Node BLE sử dụng chip ESP32-C3	20
3.2	Thiết kế Directional Coupler	21
3.2.1	Thiết kế Schematic	21
3.2.2	Thiết kế Layout	28

4	Kết quả đo đặc và phân tích	38
4.1	Kết quả của layout của một Sensor Node BLE	38
4.2	Kết quả của layout của Directional Coupler	41
	TÀI LIỆU THAM KHẢO	42
	PHỤ LỤC	42
	Phụ lục A: Mã nguồn	42

Danh sách hình vẽ

2.1	BLE channels.	2
2.2	Bluetooth LE stack architecture ^[2]	4
2.3	Link layer state transitions ^[2]	5
2.4	Sơ đồ khối cấu trúc tổng quan của ESP32-C3 ^[3]	7
2.5	Các cấu hình của Directional Coupler	9
2.6	Cấu trúc của Directional Coupler.	10
3.1	Hình ảnh cầu DHT11.	14
3.2	Typical Application.	15
3.3	Data "0"indication.	16
3.4	Data "1"indication.	16
3.5	Hình ảnh thực tế và kết quả kiểm tra đoạn code trên DEVKIT.	19
3.6	Schematic of a BLE node using ESP32-C3.	20
3.7	Schematic sơ bộ của Coupled Line Directional Coupler.	22
3.8	Giao diện của LineCalc.	23
3.9	Quy chuẩn mạch in của Thiên Lam PCB.	24
3.10	Phép đồng FR4.	25
3.11	Datasheet của FR4-Tg135.	25
3.12	Thông số độ điện thẩm và hằng số tổn hao.	26
3.13	Các thông số kích thước đường dây sau khi chạy công cụ LineCalc.	26

3.14 Schematic sau khi đã nhập thông số kích thước của microstrip.	27
3.15 Kết quả mô phỏng.	28
3.16 Layout của Coupled Line Directional Coupler.	29
3.17 Thông số mô phỏng cho substrate của PCB.	29
3.18 Kết quả sau khi chạy mô phỏng layout của Coupled Line Directional Coupler.	30
3.19 Schematic với các công cụ Optimization.	31
3.20 Khối VAR tạo các biến.	31
3.21 Khối GOAL đặt mục tiêu cần đạt được sau tối ưu.	32
3.22 Khối OPTIM để thực hiện quá trình tối ưu.	33
3.23 Kết quả sau khi tối ưu hóa.	34
3.24 Kết quả mô phỏng Schematic sau khi tối ưu hóa.	35
3.25 Layout với các thông số kích thước tối ưu hoá.	36
3.26 Kết quả mô phỏng Layout sau khi tối ưu hoá.	37
4.1 PCB tổng quát của mạch.	38
4.2 Hình ảnh 3D của mạch của một Sensor Node BLE.	39
4.3 Mạch thực tế của Sensor Node BLE.	40
4.4 Hình ảnh 3D của mạch của Directional Coupler.	41

Danh sách bảng

3.1 Bảng so sánh thông số đo được và kì vọng khi mô phỏng Coupled line Directional Coupler.	28
3.2 Bảng so sánh thông số đo được và kì vọng sau khi chạy mô phỏng layout của Coupled Line Directional Coupler.	30
3.3 Bảng so sánh thông số đo được và kì vọng khi mô phỏng Coupled line Directional Coupler sau khi Optimization.	35
3.4 Bảng so sánh thông số đo được và kì vọng sau khi chạy mô phỏng layout của Coupled Line Directional Coupler sau khi Optimization.	37

List of Listings

1	Thực thi đọc dữ liệu từ sensor DHT11.	16
2	Code thực thi chính của BLE sensor node.	43

Chapter 1. Giới thiệu tổng quan

1.1 Nhiệm vụ của nhóm

1) Đối với Sensor Node ESP32-C3

- Thiết kế schematic và layout PCB ở mức chip, bao gồm khối nguồn, ngoại vi, anten PCB và mạch matching. Phát triển firmware BLE trên nền tảng ESP-IDF, hỗ trợ các chức năng advertising, notification và quản lý năng lượng (light sleep, deep sleep, duty cycling).
- Đánh giá thực nghiệm về tiêu thụ năng lượng (dòng sleep/active), độ trễ wake-up (wake-up latency) và độ trễ vòng lặp đầu cuối (round-trip time).

2) Đối với Directional Coupler

- Thiết kế, mô phỏng cấu trúc vi dải coupled-line với hệ số ghép $-10dB$ và $-20dB$.
- Thực hiện layout PCB và chế tạo thực nghiệm.
- Đo và phân tích các tham số S (S_{11} , S_{21} , S_{31}), isolation, phase và băng thông, từ đó so sánh với kết quả mô phỏng.

3) Tương tác với nhóm khác

- Test truyền khi nối qua front-end (ĐT2) và PA (ĐT3).
- Kết nối GPIO_WU từ ĐT5 để chứng minh wake-up chain.

1.2 Cấu trúc báo cáo đề tài

Báo cáo được chia thành sáu chương chính:

Chương 1. Giới thiệu tổng quan.

Chương 2. Cơ sở lý thuyết.

Chương 3. Thiết kế và thực hiện.

Chương 4. Kết quả đo đạc và phân tích.

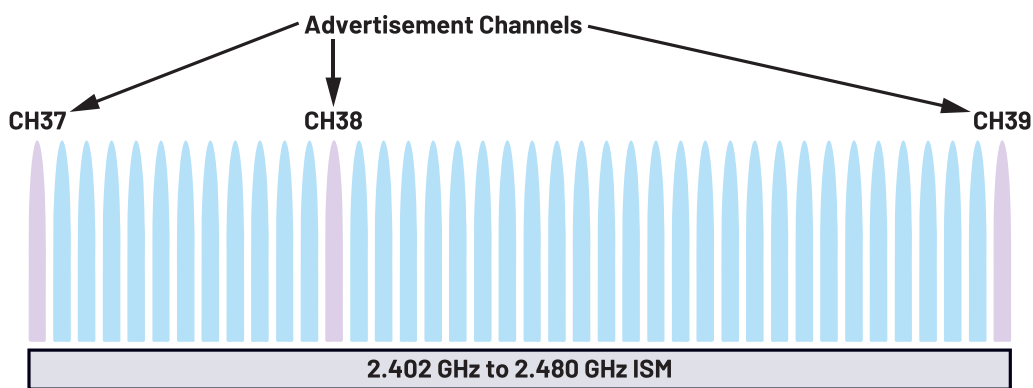
Chương 5. Thảo luận và đánh giá.

Chương 6. Kết luận và hướng phát triển.

Chapter 2. Cơ sở lý thuyết

2.1 Tổng quan về công nghệ Bluetooth Low Energy (BLE)

Bluetooth Low Energy (BLE) là một chuẩn truyền thông không dây trong băng tần ISM 2.4GHz , được thiết kế cho các ứng dụng IoT tiêu thụ năng lượng thấp. Chuẩn này sử dụng 40 kênh RF, trong đó có 3 kênh quảng bá (advertising) và 37 kênh dữ liệu, với tốc độ truyền danh định 1 Mbps và khoảng cách hoạt động khoảng 10–50 m tùy điều kiện.



Hình 2.1: BLE channels.

Các ưu điểm của BLE là:

- **Tiêu thụ điện năng thấp (Low Power Consumption):** BLE tiêu thụ ít điện năng hơn đáng kể so với **Bluetooth Classic**, khiến nó phù hợp cho các thiết bị chạy bằng pin yêu cầu thời lượng pin lâu dài.
- **Chi phí thấp hơn (Lower Cost):** Yêu cầu công suất thấp hơn của **BLE** thường dẫn đến chi phí thấp hơn để triển khai công nghệ này trong các thiết bị.
- **Kết nối được đơn giản hóa (Simplified Connectivity):** BLE đơn giản hóa quy trình khám phá thiết bị và thiết lập kết nối, giúp các thiết bị dễ dàng kết nối và giao tiếp với nhau hơn.
- **Thiết lập kết nối nhanh hơn (Faster Connection Establishment):** BLE thường thiết lập kết nối nhanh hơn **Bluetooth Classic**, từ đó giảm độ trễ trong truyền dữ liệu.
- **Khả năng tương thích với thiết bị di động (Compatibility with Mobile Devices):** BLE rất phù hợp để tích hợp với điện thoại thông minh và máy tính bảng, cho

phép giao tiếp liền mạch giữa các thiết bị và ứng dụng di động.

Điểm khác biệt quan trọng giữa BLE và Bluetooth Classic là cơ chế tiết kiệm năng lượng. BLE chỉ phát gói tin ngắn trong quá trình quảng bá và duy trì trạng thái ngủ phần lớn thời gian, thay vì giữ kết nối liên tục. Điều này giúp thiết bị chạy bằng pin nhỏ có thể hoạt động trong nhiều tháng hoặc thậm chí nhiều năm.

Kiến trúc giao thức BLE gồm hai phần: Controller (lớp vật lý và liên kết, xử lý kết nối ở mức thấp) và Host (các giao thức cao hơn như ATT, GATT, GAP và quản lý bảo mật). Giao diện giữa hai phần được chuẩn hóa thông qua Host Controller Interface (HCI), cho phép linh hoạt trong phát triển phần cứng và firmware.

Trong hoạt động, thiết bị BLE có hai chế độ chính: advertising (phát gói tin để thông báo sự hiện diện, cho phép quét và thiết lập kết nối) và connection (trao đổi dữ liệu dựa trên profile GATT). Ngoài ra, thiết bị có thể ở trạng thái chờ (idle) hoặc deep-sleep, đảm bảo mức tiêu thụ dòng cực thấp.

2.1.1 BLE Beacons

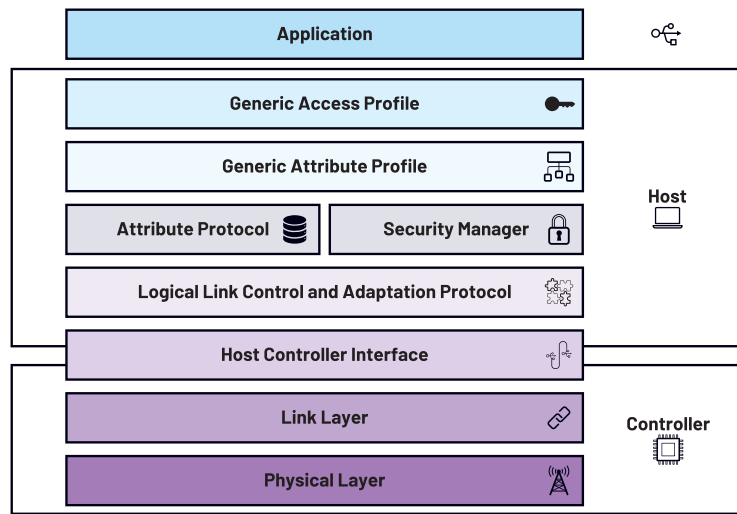
Các Bluetooth beacons là những bộ phát nhỏ, không dây và sử dụng pin, hoạt động dựa trên BLE protocol để truyền dữ liệu. Điểm đặc biệt của beacons là chúng thường hoạt động theo một chiều, tức là chúng broadcast dữ liệu đến các thiết bị Bluetooth Low Energy ở gần, nhưng không nhận dữ liệu ngược lại. Các BLE beacons không cần internet connection để thực hiện việc broadcasting này^[1].

Thông thường, điều này có thể được hình dung như một beacon duy nhất broadcast dữ liệu đến các thiết bị thông minh ở gần như cell phones, smart watches, v.v.

Hiện nay, smartphone support cho Bluetooth beacons vẫn còn hạn chế, vì vậy việc triển khai thực tế vẫn còn rất ít. Trong tương lai, chúng ta có thể sẽ thấy beacons được ứng dụng trong các lĩnh vực như proximity marketing, indoor navigation, và asset tracking trong logistics industry.

2.1.2 BLE protocol stack

Trong Bluetooth protocol stack được chia làm 3 thành phần chính hoặc là các phân tử của hệ thống như sau: **application**, **host** and **controller**. Bên trong mỗi lớp đó có các lớp riêng biệt khác trong đó.

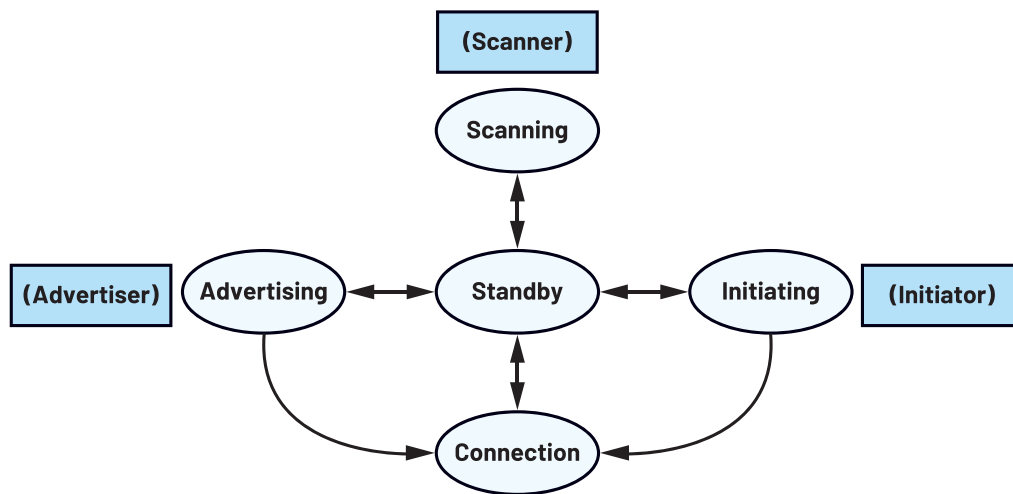


Hình 2.2: Bluetooth LE stack architecture^[2].

1) Controller (lowest layer of stack)

Controller Layer gồm 2 lớp:

- *The Physical Layer (PHY)*: là vị trí thấp nhất của BLE stack và chịu trách nhiệm cho việc truyền và nhận thực tế của tín hiệu từ không khí. Physical Layer hoạt động ở dải tần 2.4GHz ISM, và sử dụng Gaussian frequency shift keying (GFSK) để điều chế. Cơ chế sử dụng GFSK giúp cho việc truyền dữ liệu hiệu quả bằng cách nhờ chuyển đổi tần số của tín hiệu sóng mang.
- *The Link Layer (LL)*: là lớp chịu trách nhiệm cho scanning, advertising, creating, và maintaining links hoặc quản lý kết nối giữa các thiết bị với nhau. Link Layer quản lý cách chọn tần số cho việc truyền dữ liệu, sử dụng phổ lan truyền tần số để giảm thiểu nhiễu. Link Layer có các trạng thái khác nhau như:
 - Standby: là trạng thái idle khi mà không có các gói nào được nhận hoặc truyền.
 - Advertising: là trạng thái truyền gói Advertising trong khi lắng nghe các thiết bị khác yêu cầu thêm thông tin.
 - Scanning: là trạng thái Link Layer (acting as a scanner) lắng nghe các gói Advertisers và có thể yêu cầu thêm thông tin từ chúng.
 - Initiating: là trạng thái Link Layer (acting as a initiator) lắng nghe các gói đến từ Advertiser và đáp lại các gói đó bằng cách bắt đầu kết nối.
 - Connection: là trạng thái mà Link Layer đã kết nối với các Link Layer của các thiết bị BLE khác.

Hình 2.3: Link layer state transitions^[2].

2) Host Controller Interface (HCI)

HCI Layer cung cấp một kết nối giữa hardware (PHY và LL) với các layer cao hơn. Nó cho phép host (e.g., a smartphone or computer) điều khiển Bluetooth hardware.

3) Host (upper layer protocol and runs the following services)

Host Layer là lớp chứa các lớp thấp hơn của BLE stack, bao gồm:

- *Logical Link Control and Adaptation Protocol (L2CAP)*: L2CAP nằm phía trên lớp HCI và xử lý phân đoạn gói và lắp ráp lại, cũng như các luồng dữ liệu ghép kênh.
- *Security Manager Protocol (SMP)*: Lớp này xử lý các khía cạnh bảo mật như ghép nối, mã hóa và xác thực giữa các thiết bị.
- *Attribute Protocol (ATT)*: ATT quản lý truyền dữ liệu và tổ chức trong các thiết bị Bluetooth LE. Nó chịu trách nhiệm đọc, viết và khám phá các thuộc tính (ví dụ: dịch vụ và đặc điểm) trên thiết bị.
- *Generic Attribute Profile (GATT)*: GATT là một hồ sơ được xây dựng trên đầu ATT và xác định cách trao đổi dữ liệu bằng các thuộc tính.
- *Generic Access Profile (GAP)*: GAP xác định cách các thiết bị BLE tương tác ở cấp độ cơ bản, bao gồm khám phá thiết bị, thiết lập kết nối và quảng cáo.

4) Application Layer

Application Layer là nơi thực hiện các chức năng đặc biệt của BLE-enable devices. Nó tương tác với các lớp thấp hơn trong stack thông qua Generic Attribute Profile (GATT), được sử dụng để định nghĩa các services (dịch vụ), characteristic (đặc tính), and the corresponding data (dữ liệu tương ứng).

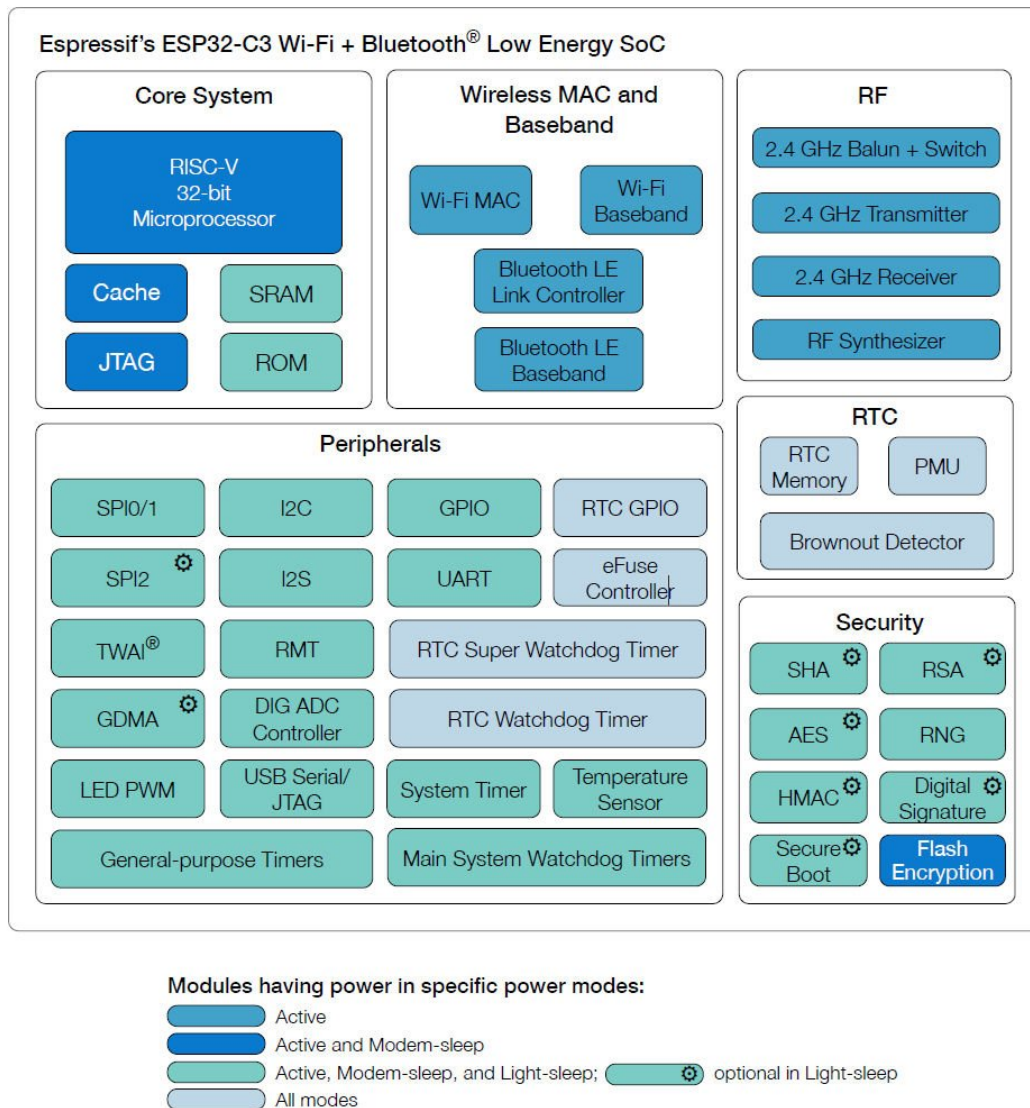
Tại Application Layer, các đặc tính và hành vi của thiết bị đã được thiết kế. Nó bao gồm các định nghĩa về services, characteristics, đặc biệt là các dữ liệu trao đổi với nhau, và thực hiện các logic cho xử lý các sự kiện như connections, disconnections, and data updates. Về bản chất, Application Layer tùy chỉnh các BLE stack cho các yêu cầu cụ thể của thiết bị với các trường hợp dự định của nó.

2.1.3 BLE working modes

- 1) **Master mode:** Module Bluetooth Low Energy của Feasycom hỗ trợ Master mode, cho phép tìm kiếm và kết nối các thiết bị Slave xung quanh. Ở chế độ này, module có thể gửi và nhận dữ liệu, đồng thời thiết lập địa chỉ MAC mặc định để tự động kết nối với Slave khi được cấp nguồn.
- 2) **Slave mode:** Trong chế độ này, Bluetooth module chỉ có thể bị phát hiện bởi thiết bị Master (không thể chủ động tìm kiếm). Sau khi kết nối, nó có thể truyền và nhận dữ liệu với host device.
- 3) **Broadcast mode:** Ở chế độ này, module thực hiện broadcast một-nhiều. Người dùng có thể cấu hình dữ liệu broadcast thông qua AT command, và module hoạt động ở chế độ tiêu thụ năng lượng thấp. Ứng dụng gồm: beacons, billboards, indoor positioning, asset tracking,... Ví dụ module: FSC-BT616, FSC-BT630 (Bluetooth 5.0); FSC-BT909 (Bluetooth 4.2).
- 4) **Mesh network mode:** Ở chế độ Mesh, nhiều module có thể liên kết thành mạng bằng star network hoặc relay technology, cho phép các mạng khác nhau kết nối với nhau. Cuối cùng, nhiều module Bluetooth có thể được quản lý và điều khiển qua smartphone hoặc tablet.

2.2 Kiến trúc và đặc điểm kỹ thuật của ESP32-C3

ESP32-C3 là vi mạch SoC do Espressif phát triển, tích hợp nhân xử lý RISC-V 32-bit, xung nhịp 160 MHz và bộ nhớ SRAM 400 KB. Điểm mạnh quan trọng của C3 là hỗ trợ Bluetooth Low Energy 5.0 cùng Wi-Fi 2.4GHz, cho phép lựa chọn linh hoạt giữa truyền tải cự ly ngắn, tiêu thụ năng lượng thấp và kết nối Internet dung lượng lớn.



Hình 2.4: Sơ đồ khối cấu trúc tổng quan của ESP32-C3^[3].

Trong bối cảnh đề tài, BLE 5.0 là thành phần trọng tâm, đảm bảo khả năng quảng bá, kết nối và truyền dữ liệu cảm biến với mức tiêu thụ năng lượng tối thiểu. ESP32-C3 cung cấp nhiều chế độ tiết kiệm năng lượng (light-sleep, deep-sleep), với dòng tiêu thụ trong deep-sleep chỉ vài μA , đáp ứng yêu cầu đo lường và tối ưu hóa năng lượng của hệ thống.

Bên cạnh đó, SoC này tích hợp ADC 12 bit, giao tiếp chuẩn SPI/I²C/UART, cho phép kết nối trực tiếp với các cảm biến nhiệt độ, độ ẩm và ánh sáng. Nguồn cấp được quản lý thông qua LDO và mạch decoupling nhằm đảm bảo ổn định cho hoạt động của RF và khối xử lý.

Về phần mềm, ESP32-C3 được hỗ trợ bởi ESP-IDF, cung cấp sẵn stack BLE (GATT, GAP) và công cụ quản lý năng lượng. Điều này giúp nhóm tập trung phát triển firmware cho

các chức năng chính: quảng bá dữ liệu, gửi thông báo định kỳ, xử lý ngắt từ GPIO wake-up, và đo lường độ trễ wake-up đến khi phát gói BLE đầu tiên.

2.3 Quản lý năng lượng trong BLE node

1) **Chiến lược tiết kiệm năng lượng:** BLE node có thể giảm đáng kể mức tiêu thụ điện thông qua:

- Duty cycling và sleep modes: node chỉ hoạt động trong chu kỳ cần thiết, còn lại ở chế độ deep sleep hoặc standby để giảm dòng rò^[4].
- Adaptive transmission power control: điều chỉnh công suất phát dựa trên khoảng cách đến thiết bị nhận, nhằm tối ưu giữa độ tin cậy liên kết và tiêu thụ năng lượng^[5].
- Dynamic scheduling và advertising interval optimization: lựa chọn khoảng thời gian advertising interval phù hợp giúp cân bằng giữa khả năng phát hiện và thời lượng pin^[6].

2) **Thu hoạch năng lượng (Energy Harvesting):** Các node BLE có thể được tích hợp các nguồn năng lượng tái tạo vi mô:

- Từ môi trường: thu năng lượng từ ánh sáng, nhiệt điện hoặc rung động cơ học.
- Mạch chuyển đổi hiệu quả: sử dụng DC-DC converter có hiệu suất cao giúp duy trì ổn định nguồn cấp.
- Cân bằng năng lượng: thiết lập cơ chế điều phối giữa năng lượng thu được và tiêu thụ, đảm bảo node hoạt động liên tục^[7].

3) **Thiết kế phần cứng tiết kiệm năng lượng:**

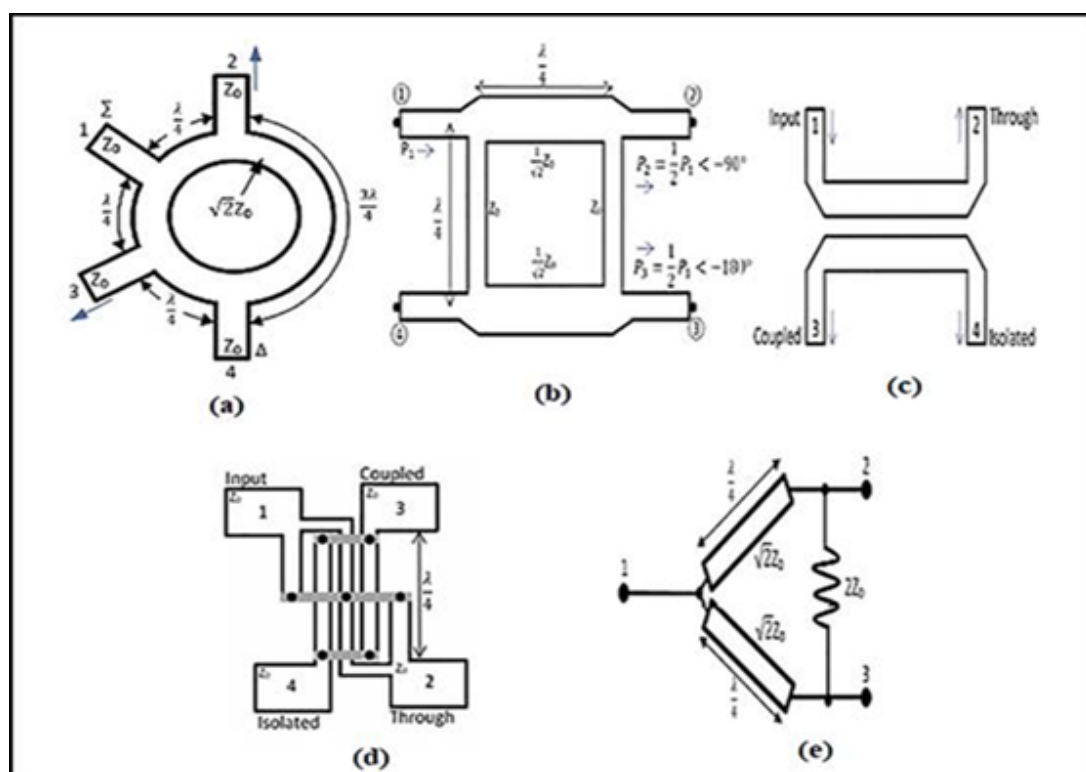
- Tối ưu thiết kế PCB để giảm tổn hao điện, kết hợp low-dropout regulator (LDO) và low-leakage capacitors.
- Cảm biến siêu tiết kiệm điện (như MEMS-based sensors) giúp giảm tiêu thụ nền.

4) **Phân tích và mô phỏng tiêu thụ năng lượng:**

- Mô hình hóa năng lượng theo trạng thái hoạt động (active, sleep, advertising, connection) cho phép ước tính chính xác thời gian hoạt động^[11].
- Đo lường dòng điện sử dụng công cụ như Nordic Power Profiler Kit II để xác định chu kỳ năng lượng.
- Tính toán giới hạn tốc độ truyền dữ liệu dựa trên dung lượng pin và hiệu suất mạch nguồn.

2.4 Lý thuyết về Directional Coupler

Directional coupler (bộ ghép định hướng) là một thành phần thụ động bốn cổng dùng để lấy mẫu (sample) năng lượng trên một đường truyền RF mà không làm nhiễu đáng kể tới đường truyền chính. Trong bối cảnh đề tài - giám sát và đo lường tín hiệu $2.4GHz$ của bộ phát BLE - dạng coupler thực thi chức năng cung cấp tín hiệu mẫu cho VNA, detector hoặc cho mạch Wake-Up Receiver, đồng thời cho phép đánh giá công suất truyền, phản sóng và phase. Phần tiếp theo trình bày những khái niệm lý thuyết cần thiết cho thiết kế microstrip coupled-line coupler, bao gồm nguyên lý hoạt động, tham số tán xạ và các đặc tính như coupling, isolation, insertion loss và phase mà nhóm tìm hiểu từ các tài liệu.

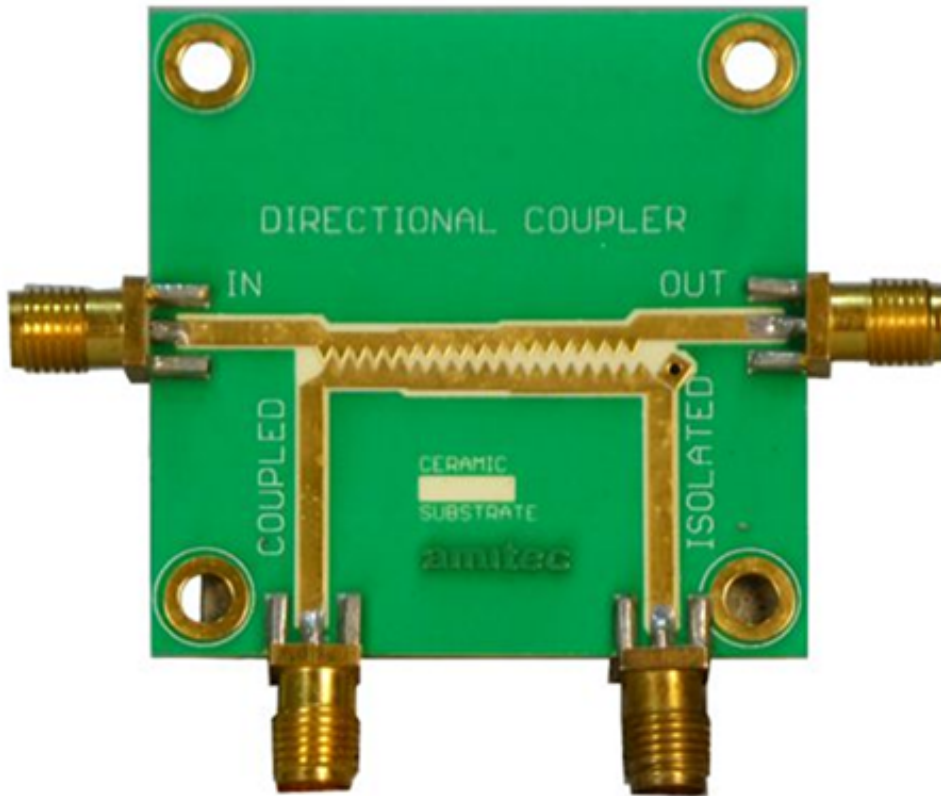


Hình 2.5: Các cấu hình của Directional Coupler: a) Ring directional coupler; b) Branch-line directional coupler; c) Coupled-line directional coupler; d) Lange directional coupler; e) Wilkinson power divider.

2.4.1 Nguyên lý vi dải và đường truyền song song (microstrip coupled-line)

Microstrip coupled-line directional coupler là cấu trúc bao gồm hai dải vi dẫn song song đặt gần nhau trên cùng một lớp dielectric với mặt đất bên dưới; tương tác giữa hai dải tạo ra hiện tượng ghép điện từ (electromagnetic coupling). Khi một sóng điện từ chạy trên dải “chính” (through line), năng lượng sẽ được truyền một phần sang dải kề bên qua tương tác trường điện (even-mode) và trường từ (odd-mode). Ở tần số trung tâm, nếu chiều dài của

đoạn ghép bằng một phần tư bước sóng dẫn ($l \approx \frac{\lambda_g}{4}$), các sóng even và odd kết hợp sao cho năng lượng được phân tách thành hai cổng: một cổng “through” (truyền chính) và một cổng “coupled” (lấy mẫu) theo chiều định hướng mong muốn; cổng thứ tư (isolated) lý tưởng sẽ có năng lượng bằng 0.



Hình 2.6: Cấu trúc của Directional Coupler.

Phân tích thường bắt đầu bằng các mode đối xứng (even) và phản đối xứng (odd). Mỗi mode có trở kháng đặc trưng khác nhau, gọi là Z_{0e} (even-mode characteristic impedance) và Z_{0o} (odd-mode characteristic impedance). Đối với mạch ghép cân bằng trên trở kháng hệ thống Z_{0o} , các quan hệ cơ bản là:

$$Z_0 = \sqrt{Z_{0e}Z_{0o}}$$

Mối liên hệ giữa trở kháng mode và độ ghép (coupling coefficient) thường được định nghĩa bằng hệ số:

$$k = \frac{Z_{0e} - Z_{0o}}{Z_{0e} + Z_{0o}}$$

Giá trị k xác định mức độ ghép điện áp giữa hai dải. Việc chọn chiều rộng dải, khoảng cách giữa hai dải và chiều dày/ ϵ_r của substrate ảnh hưởng trực tiếp tới Z_{0e} và Z_{0o} , từ đó quyết định coupling mong muốn.

Trong thiết kế thực tế, để đạt coupling cố định tại tần số trung tâm người ta điều chỉnh các thông số hình học của coupled-line; để mở rộng băng thông, cần dùng nhiều đoạn ghép nối tiếp (multi-section) hoặc kỹ thuật bù dispersion.

2.4.2 Tham số tán xạ (*S*-parameters)

Bộ ghép định hướng được mô tả hiệu quả bằng ma trận tham số tán xạ S bốn cổng. Với qui ước:

- Cổng 1 là input,
- Cổng 2 là through,
- Cổng 3 là coupled,
- Cổng 4 là isolated,

ma trận S của một coupler lý tưởng (mất mát bằng không, bắt khớp và cách ly hoàn hảo) có dạng:

$$S = \begin{bmatrix} S_{11} & \cdots & S_{14} \\ \vdots & \ddots & \vdots \\ S_{41} & \cdots & S_{44} \end{bmatrix}$$

Trong đó, mỗi phần tử S_{ij} (tất cả các cổng khác được ghép với tải chuẩn) được định nghĩa:

$$S_{ij} = \frac{V_i^-}{V_j^+}$$

với i là ngõ vào, j là ngõ ra; V^- : biên độ sóng phản xạ, V^+ : biên độ sóng tới.

2.4.3 Các hệ số đặc trưng

- **Reflection Coefficients:** $S_{11} = S_{22} = S_{33} = S_{44}$ (khớp)
- **Transmission Coefficients:** $S_{12}, S_{34}, S_{43}, S_{21} = T$ (through coefficient)
- **Coupling Coefficients:** $S_{13}, S_{23}, S_{32}, S_{31} = C$ (coupled coefficient)
- **Isolation Coefficients:** $S_{14}, S_{24}, S_{42}, S_{41} = 0$ (isolated port, lý tưởng là không có năng lượng)

Với mạch lý tưởng và mất mát bằng không, năng lượng bảo toàn đòi hỏi:

$$|S_{21}|^2 + |S_{31}|^2 = 1$$

Trong thực tế, các thành phần tổn hao (conductor loss, dielectric loss, phasor mismatch) làm cho tổng trên nhỏ hơn 1; đồng thời $S_{41} \neq 0 \Rightarrow$ isolation hữu hạn.

Các đại lượng S thường đo bằng VNA và được biểu diễn dưới dạng biên độ (dB) và phase (deg).

2.4.4 Định nghĩa thực dụng

- Coupling Coefficients (Hệ số ghép):

$$C = 10 \log \left(\frac{P_{input}}{P_{coupled}} \right) = -20 \log_{10} |S_{13}| \text{ [dB]}$$

- Directivity (Độ định hướng):

$$D = 10 \log \left(\frac{P_{coupled}}{P_{isolated}} \right) = 20 \log_{10} \left| \frac{S_{13}}{S_{14}} \right| \text{ [dB]}$$

- Isolation (Độ cách ly):

$$I = 10 \log \left(\frac{P_{input}}{P_{isolated}} \right) = -20 \log_{10} |S_{14}| = C + D \text{ [dB]}$$

- Insertion Loss (Suy hao chèn):

$$IL = 10 \log \left(\frac{P_{input}}{P_{output}} \right) = -20 \log_{10} |S_{12}| \text{ [dB]}$$

Do đồng thời biên độ và pha của S_{21} và S_{31} cho phép xác định coupler hoạt động như *quadrature hybrid* hay coupler đơn thuần. Với coupler vi dải chuẩn $\ell = \lambda_g/4$, thường thu được quan hệ pha $\approx 90^\circ$ tại tần số trung tâm.

2.4.5 Đặc tính thiết kế

- **Coupling**: ví dụ -10 dB, -20 dB, xác định tỉ lệ năng lượng lấy mẫu.

$$C = -20 \log_{10} |k|, \quad k = \frac{Z_{0e} - Z_{0o}}{Z_{0e} + Z_{0o}}$$

- **Isolation**: cần > 20 – 30 dB để tránh nhiễu ngược. Phụ thuộc vào độ đối xứng cấu trúc và sai số chế tạo.
- **Insertion Loss (IL)**: phản ánh tổn hao dẫn, dielectric, sai lệch trở kháng. IL càng nhỏ càng tốt.
- **Phase**: pha giữa tín hiệu through và coupled $\approx 90^\circ$ tại f_0 , nhưng thay đổi theo tần số.

2.4.6 Băng thông và đa đoạn

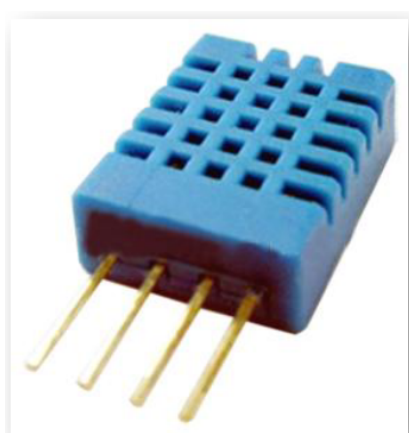
Một cặp dải ghép đơn ($\lambda_g/4$) thường có băng thông hạn chế quanh tần số thiết kế. Để mở rộng băng thông và cải thiện độ phẳng về coupling, có thể dùng multi-section coupled line hoặc kỹ thuật bù.

Chapter 3. Thiết kế và thực hiện

3.1 Thiết kế node BLE từ chip EPS32-C3

3.1.1 Sensor

Nhóm sử dụng **DHT11 Humidity & Temperature Sensor** cho Node. Với DHT11 Sensor có chức năng vừa đo độ ẩm và nhiệt độ. Thực hiện bằng giao thức one-wire serial interface.



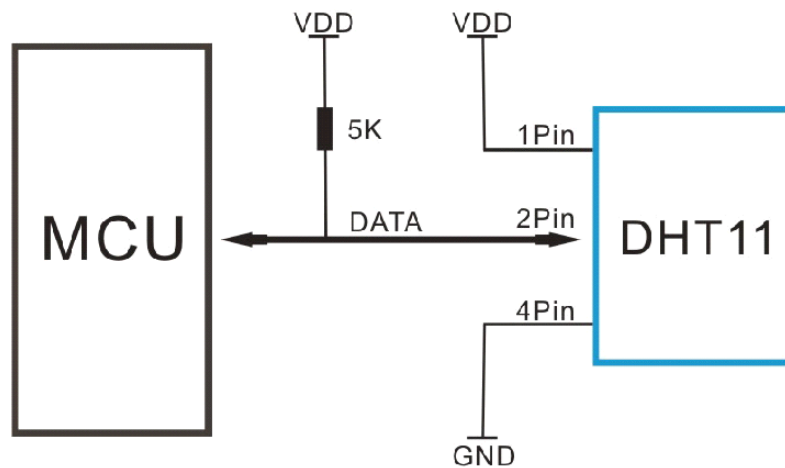
Hình 3.1: Hình ảnh cầu DHT11.

Với thiết kế nhỏ gọn, tiêu thụ công suất thấp và có thể truyền tín hiệu lên tới 20m.

Overview

Item	Measurement Range	Humidity Accuracy	Temperature Accuracy	Resolution	Package
DHT11	20 – 90% RH 0 – 50°C	±5% RH	±2°C	1	4 Pin Single Row

Typical Application



Hình 3.2: Typical Application.

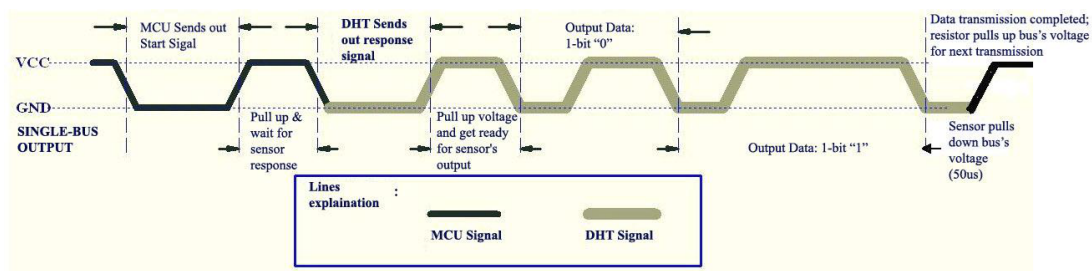
Power supply

Điện áp cấp cho DHT11 thông thường dao động từ 3 – 5.5 VDC.

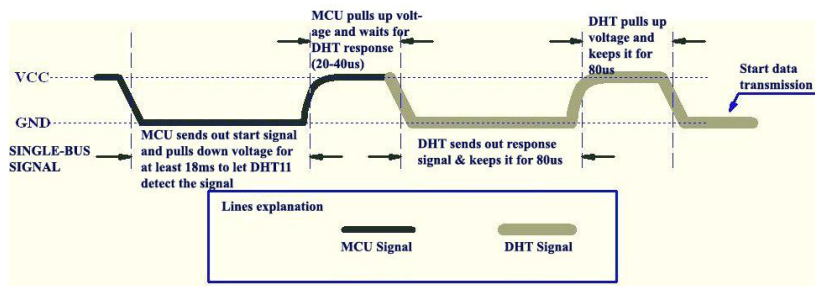
Communication Process: Serial Interface (Single-Wire Two-Way)

Định dạng dữ liệu một đường truyền được sử dụng để giao tiếp và đồng bộ giữa MCU và cảm biến DHT11. Một quá trình giao tiếp mất khoảng $4ms$.

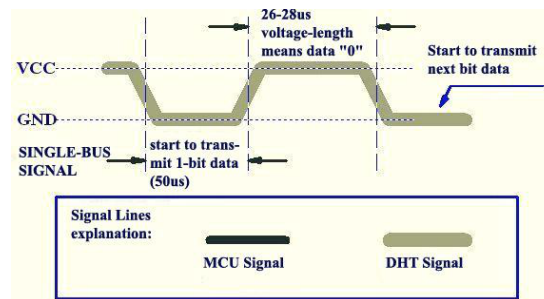
- Overall Communication Process: Khi MCU gửi tín hiệu hoạt động, DHT11 chuyển từ chế độ tiết kiệm năng lượng sang chế độ hoạt động và chờ MCU hoàn thành tín hiệu bắt đầu. Khi MCU đã hoàn thành thì DHT11 sẽ gửi một tín hiệu xác nhận với 40-bit dữ liệu bao gồm các dữ liệu liên quan đến độ ẩm và nhiệt độ cho MCU.



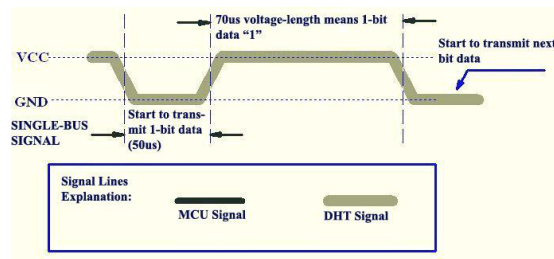
- MCU Sends out Start Signal to DHT: Nếu không hoạt động thì đường dây Data sẽ luôn ở mức cao. Nếu có tín hiệu cần giao tiếp giữa MCU và DHT11 thì MCU sẽ kéo điện áp trên đường dây Data từ 1 xuống 0 trong khoảng ít nhất $18ms$ để đảm bảo DHT nhận được tín hiệu start từ MCU, sau đó MCU sẽ kéo lên 1 trong khoảng $20 - 40\mu s$.



- DHT Responses to MCU: Khi DHT nhận được tín hiệu start, nó sẽ gửi lại một tín hiệu xác nhận low-voltage-level với khoảng 18ms. Sau đó, chương trình của DHT đặt mức điện áp của đường dữ liệu đơn từ thấp lên cao và giữ trong 80us để DHT chuẩn bị gửi dữ liệu.



Hình 3.3: Data "0" indication.



Hình 3.4: Data "1" indication.

Thực thi đọc dữ liệu từ DHT11

```

1  #include <DHT.h>
2
3  // ----- DHT11 Configuration -----
4  #define DHTPIN 10
5  #define DHTTYPE DHT11
6  DHT dht(DHTPIN, DHTTYPE);
7
8  void setup() {
9      Serial.begin(115200);
10     dht.begin();
11
12     // --- Read DHT11 sensor data ---

```

```

13     float temperature = dht.readTemperature();
14     float humidity = dht.readHumidity();
15 }

```

Listing 1: Thực thi đọc dữ liệu từ sensor DHT11.

3.1.2 BLE

Tổng quan của chương trình

- **Advertising BLE:** Là quá trình mà ESP32 phát các gói tin quảng bá định kỳ chứa thông tin cơ bản của thiết bị, chẳng hạn như tên thiết bị và UUID của dịch vụ BLE. Nhờ vào cơ chế này, các thiết bị xung quanh (ví dụ: smartphone hay laptop) có thể phát hiện và khởi tạo kết nối với ESP32.

```

1     pServer->getAdvertising()->start();
2

```

- **Notification BLE:** Là quá trình mà thiết bị đóng vai trò Server (ESP32) chủ động gửi dữ liệu cập nhật đến thiết bị Client (ví dụ: điện thoại) mà không cần Client phải gửi yêu cầu đọc. Cơ chế này rất hữu ích để truyền liên tục các giá trị cảm biến theo thời gian thực như nhiệt độ và độ ẩm.

```

1     pCharacteristicTX->notify();
2

```

- **Payload BLE:** Là phần dữ liệu thực tế được chứa trong gói tin BLE. Trong hệ thống này, payload bao gồm chuỗi dữ liệu định dạng sẵn, chứa thông tin về nhiệt độ và độ ẩm đo được từ cảm biến DHT11.

```

1     pCharacteristicTX->setValue(sensorData);
2

```

Quản lý power state (sleep, wake-up, duty cycle)

- **Sleep Mode:** Giảm tiêu thụ năng lượng thông qua.

```

1     esp_deep_sleep_start()
2

```

- **Wake-up Timer:** Tự thức dậy sau một khoảng thời gian.

```

1     esp_sleep_enable_timer_wakeup()
2

```

- **Duty Cycle Control:** Giới hạn thời gian hoạt động.

```

1  void loop() {
2      // Not used. After deep sleep, ESP32 restarts and runs setup() again.
3  }
4

```

Ngắt (interrupt) từ GPIO_WU

- **Setup GPIO:** GPIO_03 là chân nhận tín hiệu đánh thức từ bộ WuRx (Wake-up Receiver).

```

1  #define WURX_PIN    3      // External wake-up interrupt from WuRx
2

```

- **Nguyên nhân đánh thức:** Nếu ESP32-C3 được đánh thức bởi tín hiệu ngắt ngoài (EXT0), thì nguyên nhân là do WuRx kích hoạt thông qua GPIO3. Nếu không, thì là khởi động bình thường (sau reset hoặc cấp nguồn).

```

1      // --- Check wake-up reason ---
2      esp_sleep_wakeup_cause_t wakeup_reason = esp_sleep_get_wakeup_cause();
3      if (wakeup_reason == ESP_SLEEP_WAKEUP_EXT0) {
4          Serial.println("[SYSTEM] Wake-up triggered by WuRx (GPIO3)");
5      } else {
6          Serial.println("[SYSTEM] Normal power-up");
7      }
8

```

- **Kích hoạt cơ chế đánh thức:** đánh thức bằng mức tín hiệu thấp (LOW) trên GPIO_03.

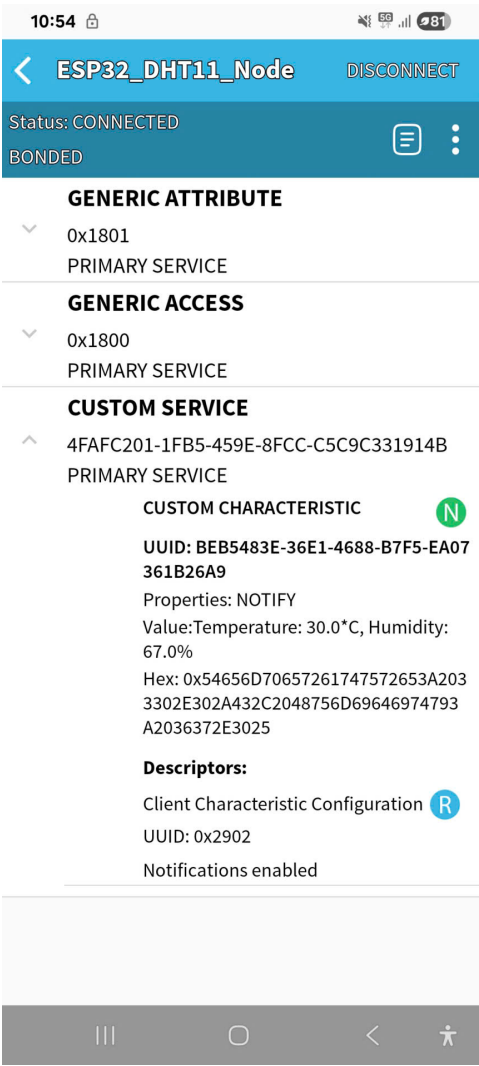
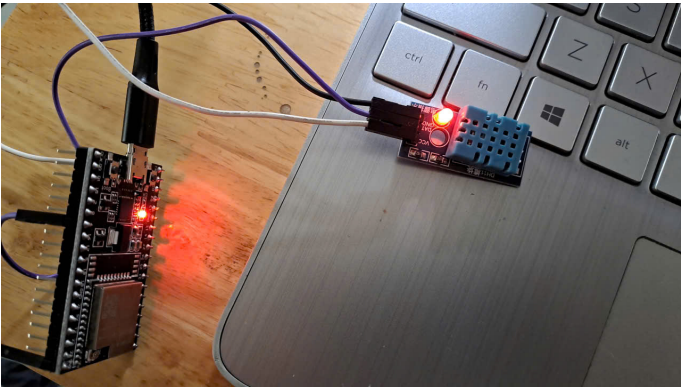
```

1      // Enable wake-up on GPIO3 (LOW level = wake-up)
2      esp_sleep_enable_ext0_wakeup((gpio_num_t)WURX_PIN, 0);
3

```

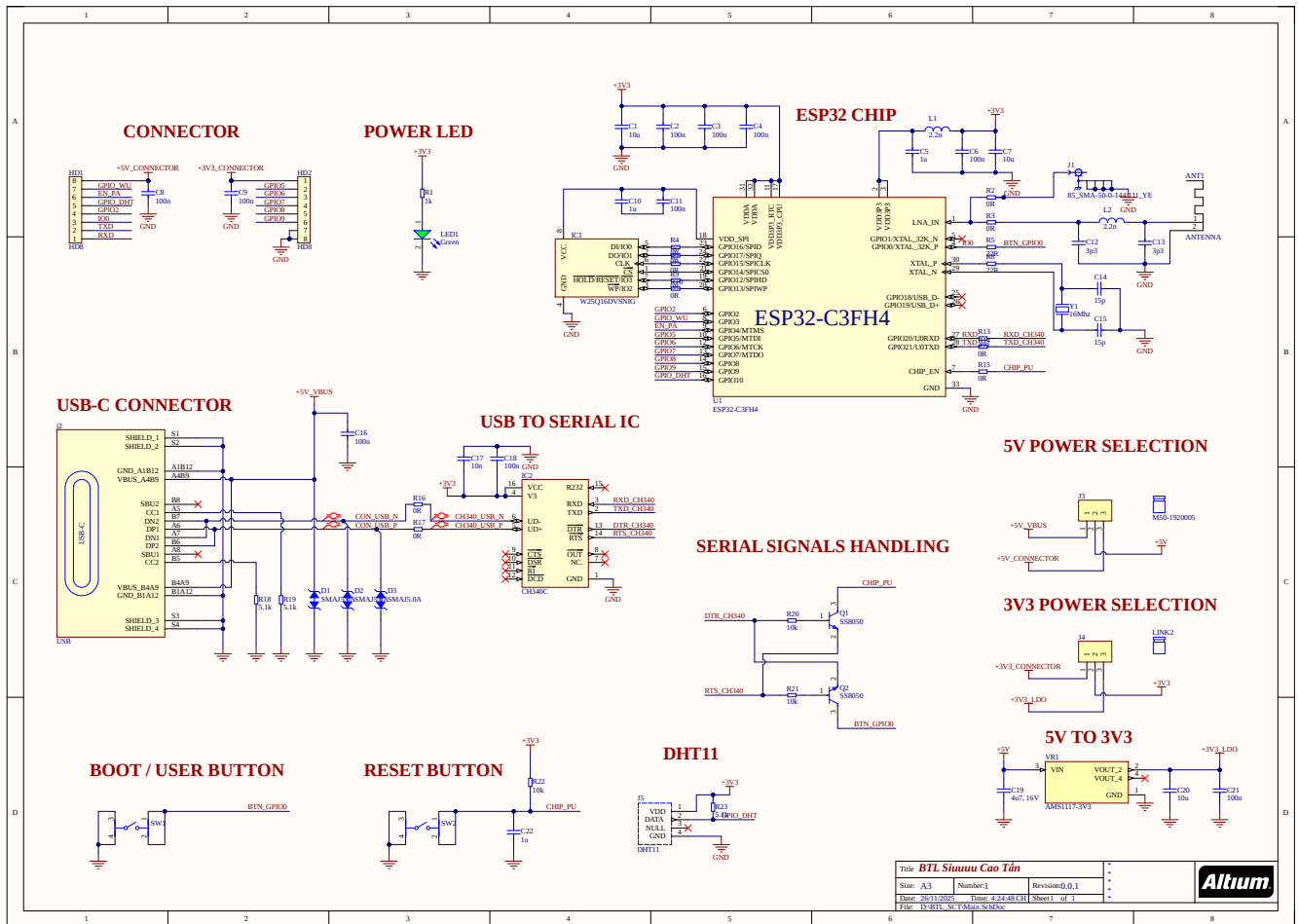
Kiểm tra chương trình trên ESP32-C3-DevKitM-1

Nhóm em sử dụng DEVKIT V1 ESP32 để kiểm tra trước chương trình vì DEVKIT trên chip ESP32-C3. Nhóm em sử dụng thêm phần mềm **BLE Scanner** () để kiểm tra dữ liệu gửi về smartphone.



Hình 3.5: Hình ảnh thực tế và kết quả kiểm tra đoạn code trên DEVKIT.

3.1.3 Sơ đồ mạch của một Sensor Node BLE sử dụng chip ESP32-C3



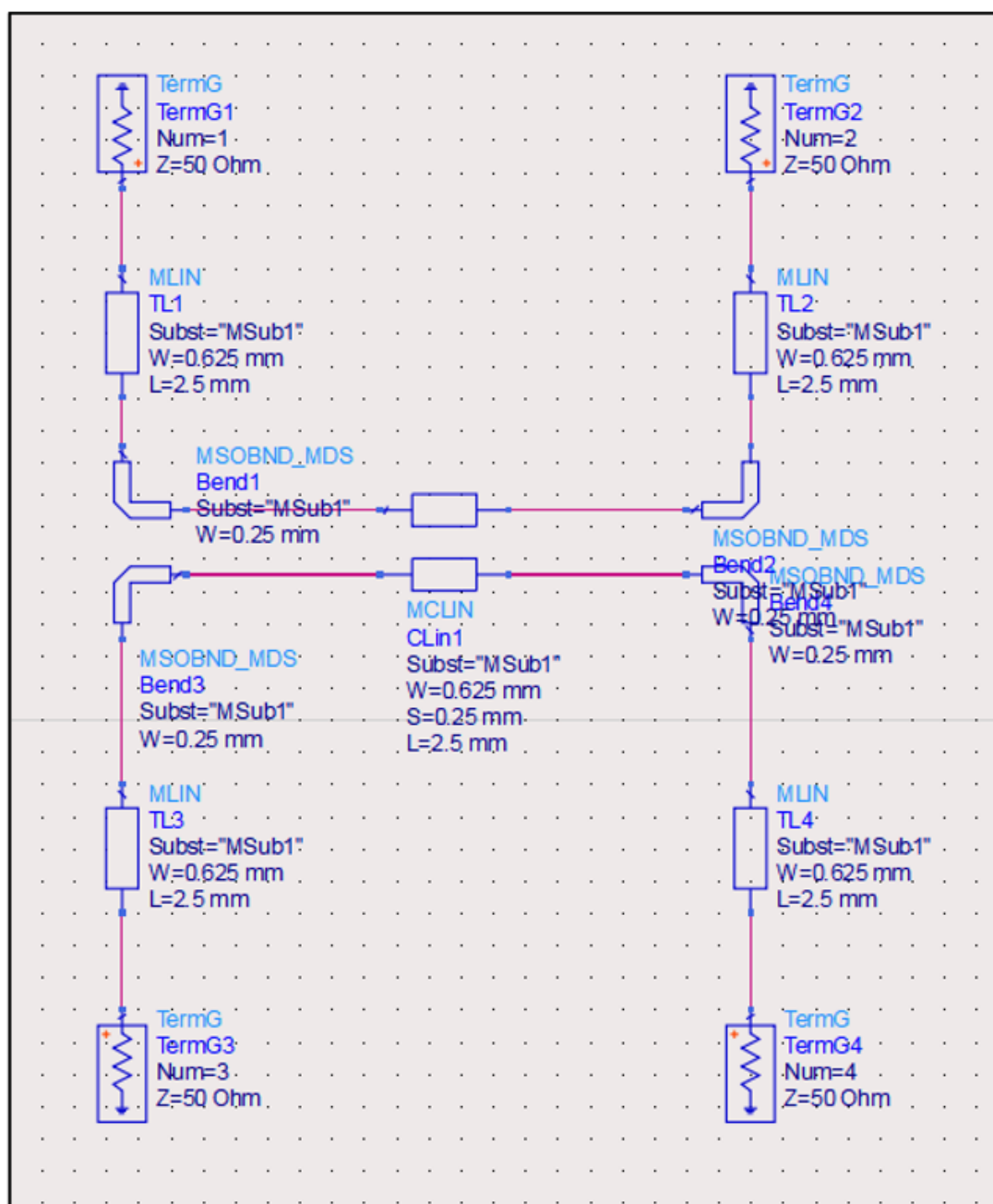
Hình 3.6: Schematic of a BLE node using ESP32-C3.

Comment	Description	Designator	Footprint	LibRef	Quantity
ANTENNA		ANT1		ANTENNA	1
10u		C1, C7, C20	1206_CAP	CAP_1206	3
100n		C2, C3, C4, C6, C8, C9, C11, C16, C17, C18, C21	1206_CAP	CAP_1206	11
1u		C5, C10, C22	1206_CAP	CAP_1206	3
3p3		C12, C13	0805_CAP	CAP_0805	2
15p		C14, C15	0805_CAP	CAP_0805	2
4u7, 16V	CAP CER 4.7UF 16V X5R 0402	C19	CAPC1005X55N	CL05A475M05NUN C	1
SMAJ5.0A	TVS DIODE 5.3VWM 12VC SOD882, TVS DIODE	D1, D2, D3	SMAJ5.0A	AQ3045-01ETG	3
HD8	Header 8	HD1, HD2	Header_8	Header8	2
W25Q16DVSNG		IC1	W25Q16DVSNG	W25Q16DVSNG	1
CH340C		IC2	CH340C	CH340C	1
USB		J2	USB	USB	1
85_SMA-50-0-144/111_YE	RF Connectors / Coaxial Connectors, Right Angle Pcb Jack HUBER+SUHNER 85_SMA-50-0-144/111_YE	J1	SMA	85_SMA-50-0-144/111_YE	1
FTS-103-01-F-S	CONN HEADER VERT 3POS 1.27MM	J3, J4	HDR_1X3_1.27MM	FTS-103-01-F-S	2
DHT11	Capacitive-type humidity and temperature module/sensor	J5	DHT11	DHT22	1
2.2n		L1, L2	0805_INDUCTOR	Inductor_0805	2
LED-0805GVC	LED Green 2mA 3V 0805	LED1	Led 0805G	LED-0805GVC	1
M50-1920005	CONN SHUNT 1.27MM RED	LINK1, LINK2		M50-1920005	2
SS8050	TRANS NPN 25V 1.5A SOT23-3	Q1, Q2	SS8050	SS8050-G	2
1K		R1	RES_0R0_1206	RES_0R0_1206	1
0R		R2, R3, R4, R6, R7, R9, R10, R11, R12, R14, R15, R16	RES_0R0_1206	RES_0R0_1206	12
22		R5, R8	RES_0R0_1206	RES_0R0_1206	2
499		R13	RES_0R0_1206	RES_0R0_1206	1
5.1K		R17, R18, R22	RES_5K1_0805	RES_0805	3
10K		R19, R20, R21	RES_0R0_1206	RES_0R0_1206	3
	SWITCH TACTILE SPST-NO 0.05A 12V	SW1, SW2	Nut nhan 4 chan	PTS645SH50SMTR 92LFS	2
ESP32-C3FH4		U1	ESP32-C3FN4	ESP32-C3FH4	1
LM1117IMP-3.3/NOPB	800-mA 15-V linear voltage regulator	VR1	LM1117 3V3	LM1117IMP-3.3/NOPB	1
40Mhz	Crystal For all Crystal 2pin.	Y1	Thach Anh 2 chan cam	Thach Anh 2 chan Cam	1

3.2 Thiết kế Directional Coupler

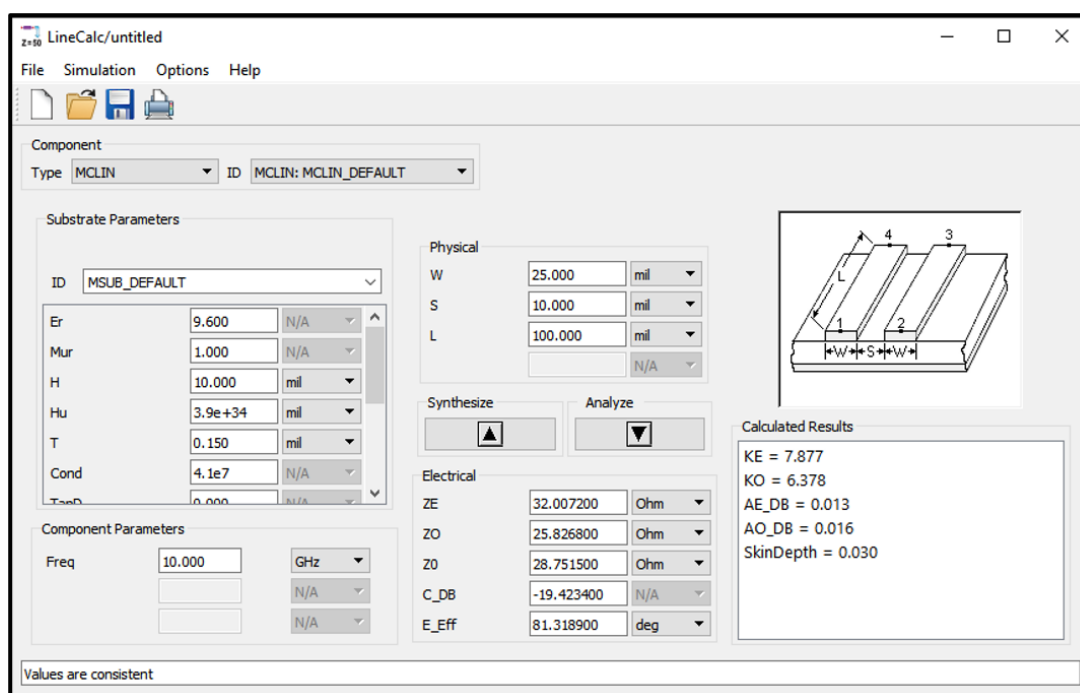
3.2.1 Thiết kế Schematic

Sử dụng phần mềm Advanced Design System của Keysight để thiết kế Schematic sơ bộ cho Coupled Line Directional Coupler.



Hình 3.7: Schematic sơ bộ của Coupled Line Directional Coupler.

Sử dụng công cụ LineCalc để tiến hành tính toán các thông số cho Microstrip:



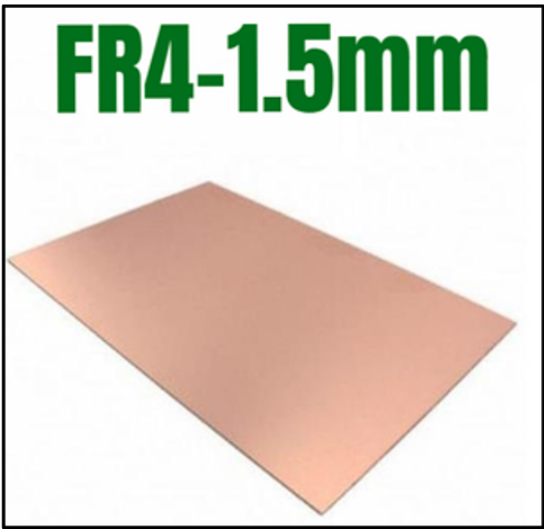
Hình 3.8: Giao diện của LineCalc.

Đầu tiên, ta sẽ tính toán thông số cho đường dây trước, để làm được thì ta sẽ xem quy chuẩn PCB của nhà sản xuất PCB trước. Ở đây nhóm chọn Thiên Lam PCB để làm mạch in PCB:

QUY CHUẨN MẠCH IN THÔNG DỤNG TẠI THIÊN LAM PCB			
Tiêu chuẩn	Giá trị	Hình Ảnh	Mô tả
Số lớp (Layer count)	1-16 lớp		Nhân xử lý board 1-16 lớp
Kích thước tấm mạch tối thiểu (min board size)	5mm x 5mm		Nếu bạn muốn sản xuất nhỏ hơn thông số này có thể thêm ở phần ghi chú hoặc email trực tiếp cho chúng tôi
Kích thước tấm mạch tối đa (max board size)	500mm x 500mm		Nếu bạn muốn sản xuất lớn hơn thông số này có thể thêm ở phần ghi chú hoặc email trực tiếp cho chúng tôi
Dung sai kích thước	±0.2mm		±0.2mm for CNC routing, and ±0.4mm for V-scoring
Vật liệu PCB (Material)	FR-4, Aluminum, FPC (mạch dẻo)		FR-4 Tg 135 / Tg140 / Tg155 / Tg170
Màu soldermask (soldermask color)	Xanh lá, đỏ, đen, trắng, vàng, tím, xanh dương.		Thiên Lam PCB sử dụng màng chống hàn LPI (Liquid Photo Imageable), giúp bảo vệ mạch tránh khỏi các tác động bên ngoài như bụi bẩn, độ ẩm và oxy hóa
Độ dày đồng	0.5 - 5oz		
Khoảng cách nhỏ nhất (minimum clearance)	0.15mm (6 mil)		Khoảng cách nhỏ nhất giữa hai vật thể, chẳng hạn như hai đường dẫn trên một mạch in hoặc giữa mạch in và các thành phần điện tử khác, để đảm bảo không có sự va chạm hoặc xung đột xảy ra trong quá trình vận hành
Độ dày tấm mạch in (board thickness)	0.4, 0.6, 0.8, 1.0, 1.2, 1.6, 2.0 mm		Độ dày của PCB thành phẩm
Độ rộng đường mạch (trace width)	0.076 mm (3 mil)		Tùy theo độ dày lớp đồng mà giá trị này khác nhau là tốt nhất: 4 mil với độ dày là 1oz. 5 mil với độ dày là 2oz. 8 mil với độ dày là 3oz.
Đường kính lỗ tối thiểu (min drill size)	0.2mm (7.9 mil)		Lỗ khoan có đường kính cơ học 0.2 mm nhưng trong thiết kế chọn giá trị từ 0.3 trở lên
Độ rộng nét tối thiểu (Minimum line width)	0.153 mm (6 mil)		Các ký tự có chiều rộng nhỏ hơn 6 mil (0.153mm) sẽ không thể nhận biết được
Độ cao chữ tối thiểu (Minimum text height)	1.0 mm (40 mil)		Chiều cao ký tự nhỏ hơn 40 mil (1.0mm) sẽ không nhận biết được. => AS-D16

Hình 3.9: Quy chuẩn mạch in của Thiên Lam PCB.

Có thể thấy Thiên Lam PCB sử dụng vật liệu PCB là FR-4 Tg135/140/155/170 với các số thể hiện nhiệt độ mà PCB chịu đựng được. Nhóm sử dụng phíp đồng FR4 độ dày 1.5mm để thực hiện PCB cho Coupler:



Hình 3.10: Phíp đồng FR4.

more than you expect



WE
WÜRTH ELEKTRONIK

Standard FR4 TG135 Datasheet

Classification according to IPC-4101 E / 21

Reinforcement: Woven E-Glass
Resin System: Epoxy, unfilled

Explanations :
C = preconditioning in humidity chamber
E = preconditioning at temperature

The figures following the letter symbols indicate with the first digit the duration of the preconditioning in hours, with the second digit the preconditioning temperature in °C and with the third digit the relative humidity.

Laminate Requirements	Thickness < 0,50mm		Thickness ≥ 0,5mm		Units	Test Method
	Typical Value	Specification	Typical Value	Specification	Metric	IPC-TM-650 or as described
Peel Strength, minimum A: Low profile copper foil and very low profile copper foil – all copper foil > 17µm B: Standard profile copper foil 1. After thermal stress 2. At 125°C 3. After process solutions C: All other foil - composite	0,9	0,70	0,95	0,70	N/mm	2.4.8
	1,05	0,80	1,20	1,05		2.4.8.2
	0,95	0,70	1,15	0,70		2.4.8.3
	0,8	0,55	1,0	0,80		2.4.8
		AABUS		AABUS		
Volume Resistivity, minimum A: C-96/35/90 B: After humidity conditioning C: At elevated temperature E-24/125	4 10 ⁶	10 ⁶	6 10 ⁶ 4 10 ⁶	10 ⁶	MΩ cm	2.5.17.1
	7 10 ⁶	10 ³	7 10 ⁶	10 ³		
Surface Resistivity, minimum A: C-96/35/90 B: After humidity conditioning C: At elevated temperature E-24/125	1 10 ⁶	10 ⁴	3 10 ⁶	10 ⁴	MΩ	2.5.17.1
	6 10 ⁶	10 ³	6 10 ⁶	10 ³		
Moisture Absorption, maximum	0,4		0,4	0,80	%	
Dielectric Breakdown, minimum			45	40	kV	2.5.6

Hình 3.11: Datasheet của FR4-Tg135.

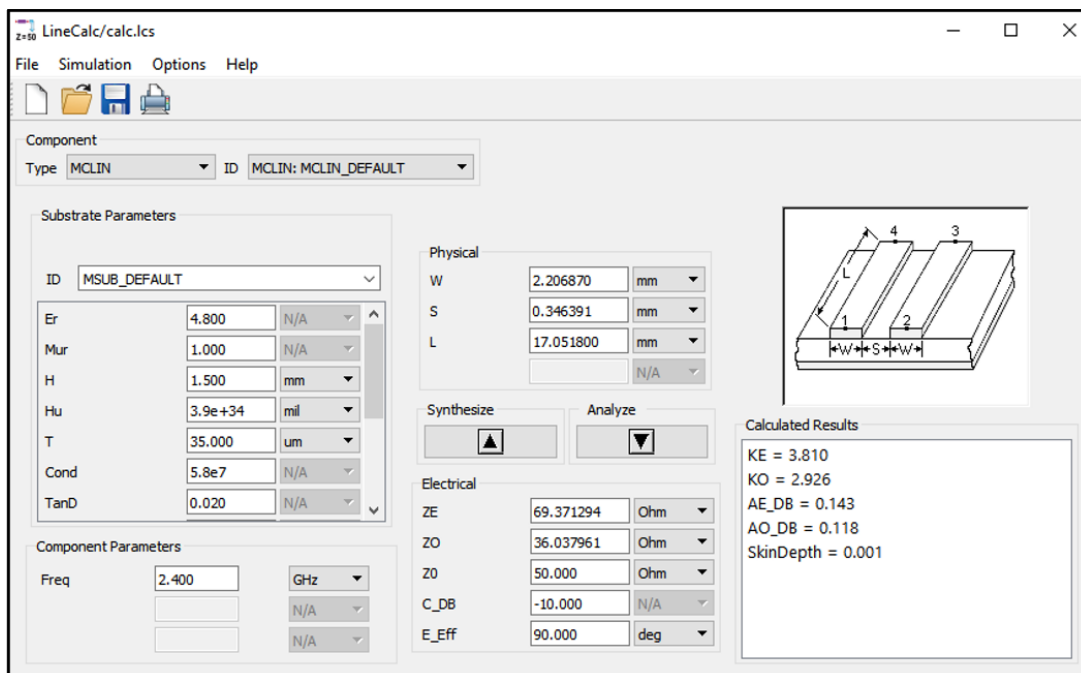
Trong datasheet này, ta chú ý 2 thông số:

Permittivity @ 1MHz (Laminate and prepreg as laminated)	4,2-4,6	5,4	4,6-4,9	5,4		2.5.5.2 2.5.5.3 2.5.5.9
Loss Tangent @ 1MHz (Laminate and prepreg as laminated)	0,015-0,02	0,035	0,015-0,02	0,035		2.5.5.2 2-5.5.3 2.5.5.9

Hình 3.12: Thông số độ điện thẩm và hằng số tổn hao.

Từ datasheet, với độ dày của board PCB là 1.5 mm nên nhóm sẽ chọn giá trị $Er = 4.8$ và $TanD = 0.02$.

Sau đó, sử dụng công cụ LineCalc của ADS để thực hiện tính toán kích thước của đường dây microstrip cho Coupler:



Hình 3.13: Các thông số kích thước đường dây sau khi chạy công cụ LineCalc.

Thông số của đường dây:

- Độ điện thẩm: $Er = 4.8$
- Độ dày của bo mạch PCB: $H = 0.6 \text{ mm}$
- Độ dày của đường microtrip: $T = 35 \mu\text{m}$
- Độ dẫn điện của microtrip: $Coud = 5.8 \times 10^7 \text{ S/m}$ (Đối với microtrip là đồng)
- Hệ số tổn hao: $TanD = 0.02$
- Điện trở của đường dây: $Z_0 = 50 \Omega$

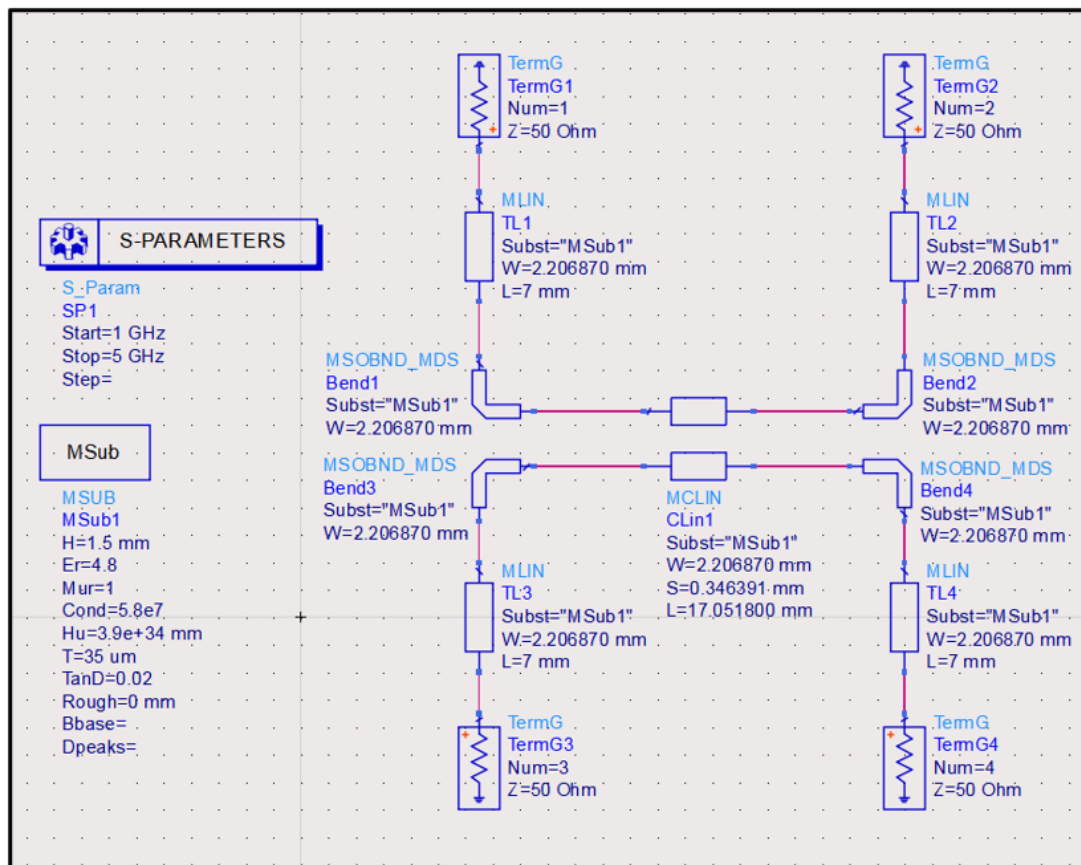
- Hệ số Coupling: $C = -10dB$ (để lấy ra 1/10 công suất ngõ vào làm tín hiệu)

Từ đó, ta có kích thước của đường dây microstrip coupling:

- Chiều rộng đường dây: $W = 2.20687 \text{ mm}$
- Chiều dài của cặp dây coupling: $L = 17.0518 \text{ mm}$
- Khoảng cách giữa 2 đường dây coupling: $S = 0.346391 \text{ mm}$

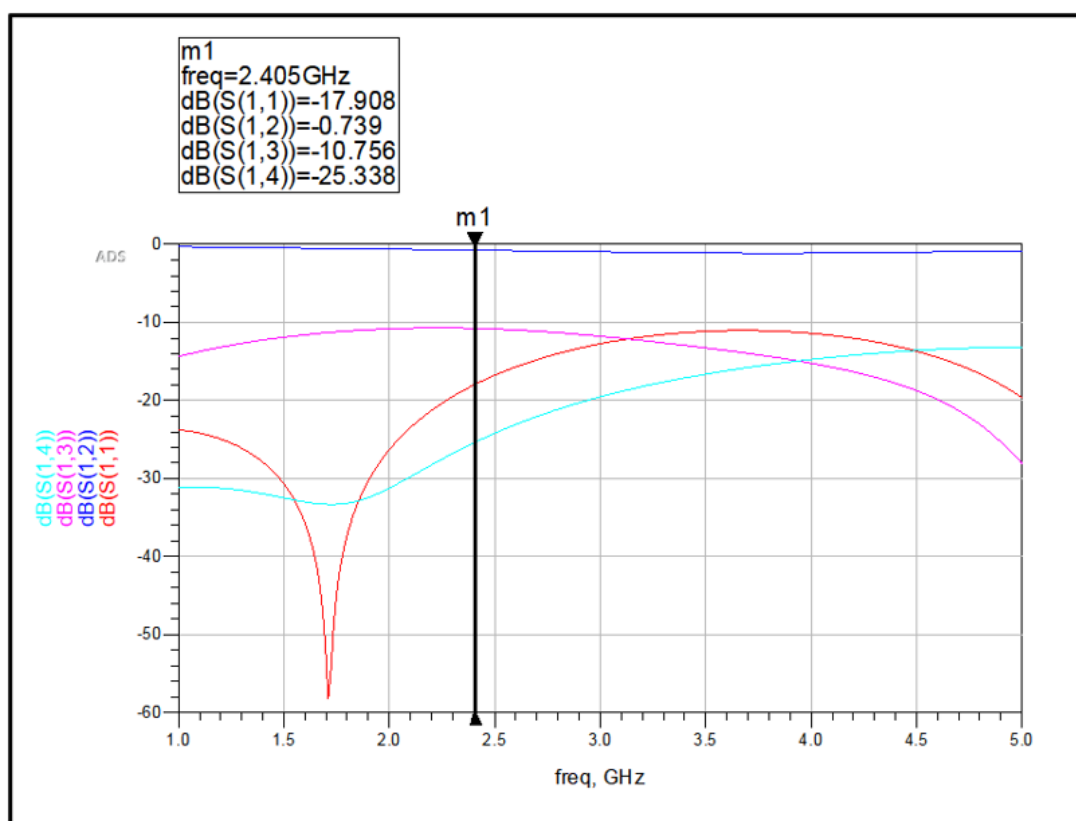
Thông số đường dây MLIN:

- Chiều rộng đường dây: $W = 2.20687 \text{ mm}$
- Chiều dài của đường dây: $L = 7 \text{ mm}$



Hình 3.14: Schematic sau khi đã nhập thông số kích thước của microstrip.

Sau đó, tiến hành chạy mô phỏng và kiểm tra kết quả:



Hình 3.15: Kết quả mô phỏng.

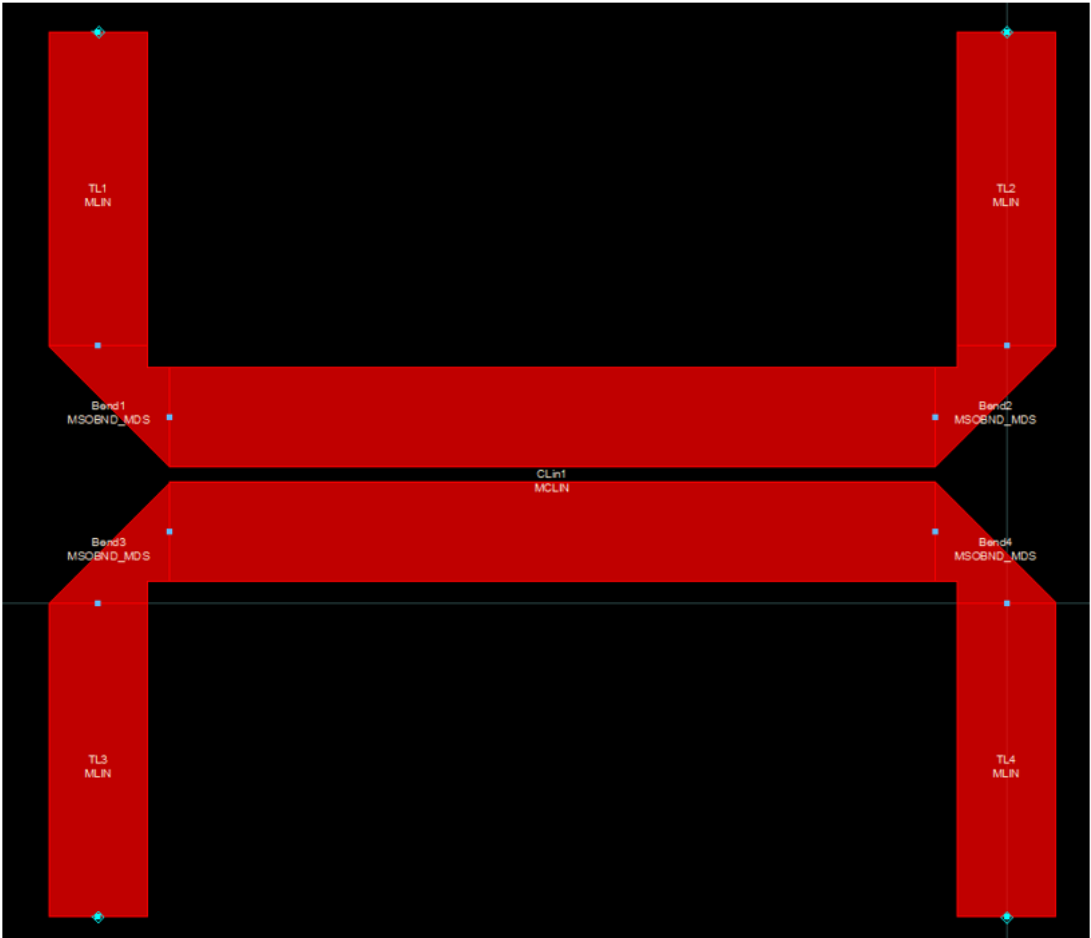
Đánh giá kết quả mô phỏng:

Thông số	Kì vọng	Đạt được	Đánh giá
S_{11}	Nhỏ hơn -20 dB	-15.806 dB	Chưa đạt
S_{12}	Lớn hơn -1 dB	-0.584 dB	Đạt
S_{13}	Gần bằng -10 dB	-10.649 dB	Đạt
S_{14}	Nhỏ hơn -20 dB	-23.589 dB	Đạt

Bảng 3.1: Bảng so sánh thông số đo được và kì vọng khi mô phỏng Coupled line Directional Coupler.

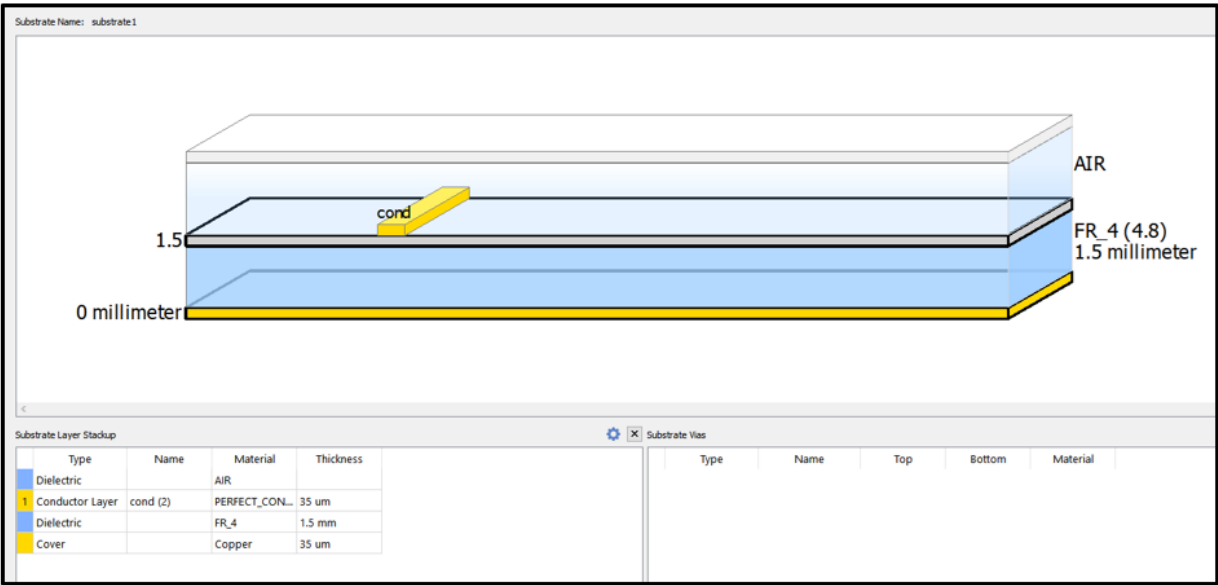
3.2.2 Thiết kế Layout

Từ những thông số, kích thước đã tính toán ở trên, ta tiến hành vẽ layout trên ADS:

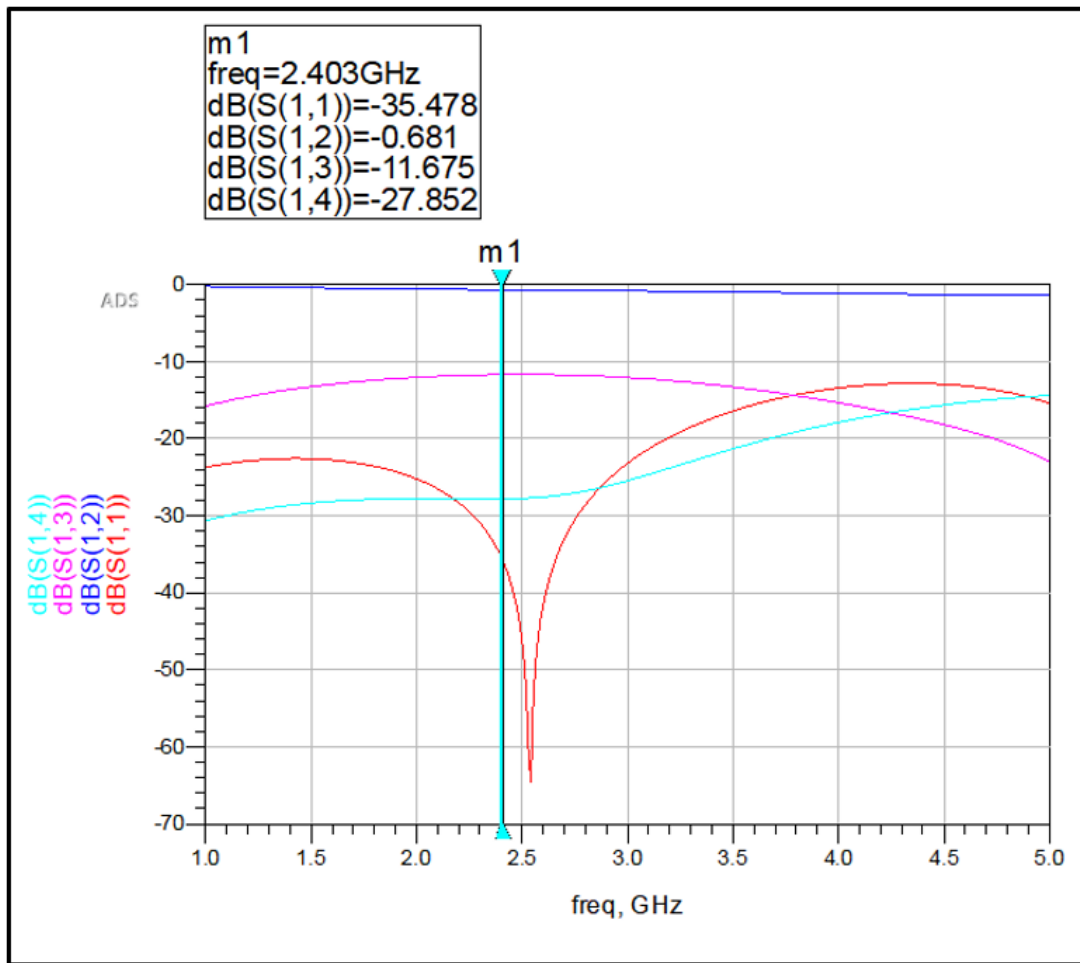


Hình 3.16: Layout của Coupled Line Directional Coupler.

Sau khi hoàn thành layout, tiến hành chạy mô phỏng và kiểm tra kết quả đạt được:



Hình 3.17: Thông số mô phỏng cho substrate của PCB.



Hình 3.18: Kết quả sau khi chạy mô phỏng layout của Coupled Line Directional Coupler.

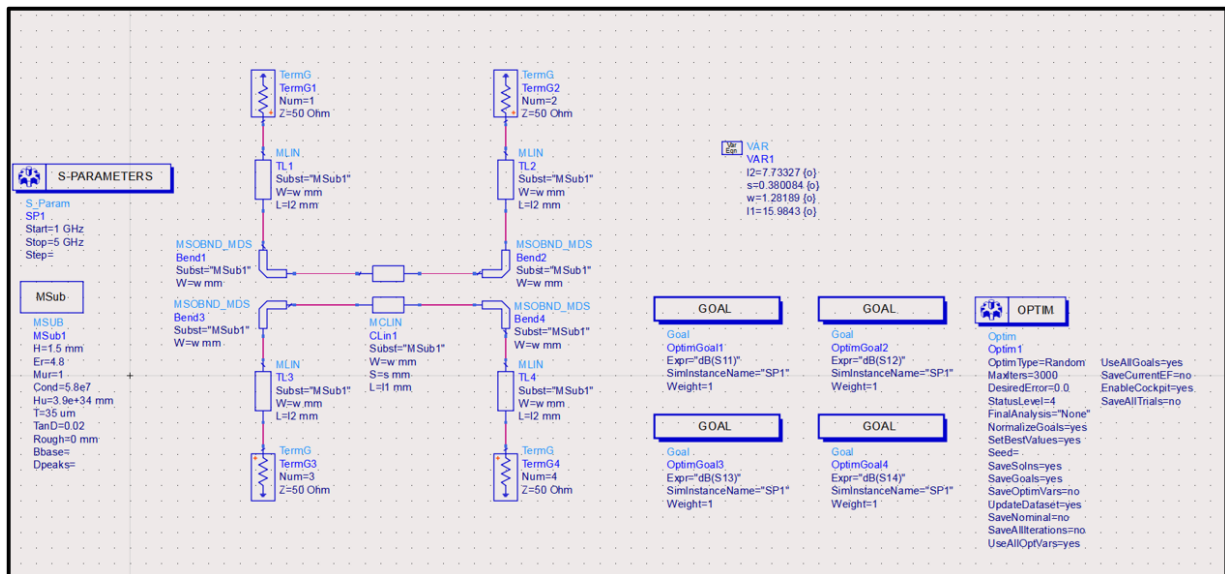
Đánh giá kết quả mô phỏng:

Thông số	Kì vọng	Đạt được	Đánh giá
S_{11}	Nhỏ hơn -20 dB	-35.478 dB	Đạt
S_{12}	Lớn hơn -1 dB	-0.681 dB	Đạt
S_{13}	Gần bằng -10 dB	-11.675 dB	Gần đạt
S_{14}	Nhỏ hơn -20 dB	-27.852 dB	Đạt

Bảng 3.2: Bảng so sánh thông số đo được và kì vọng sau khi chạy mô phỏng layout của Coupled Line Directional Coupler.

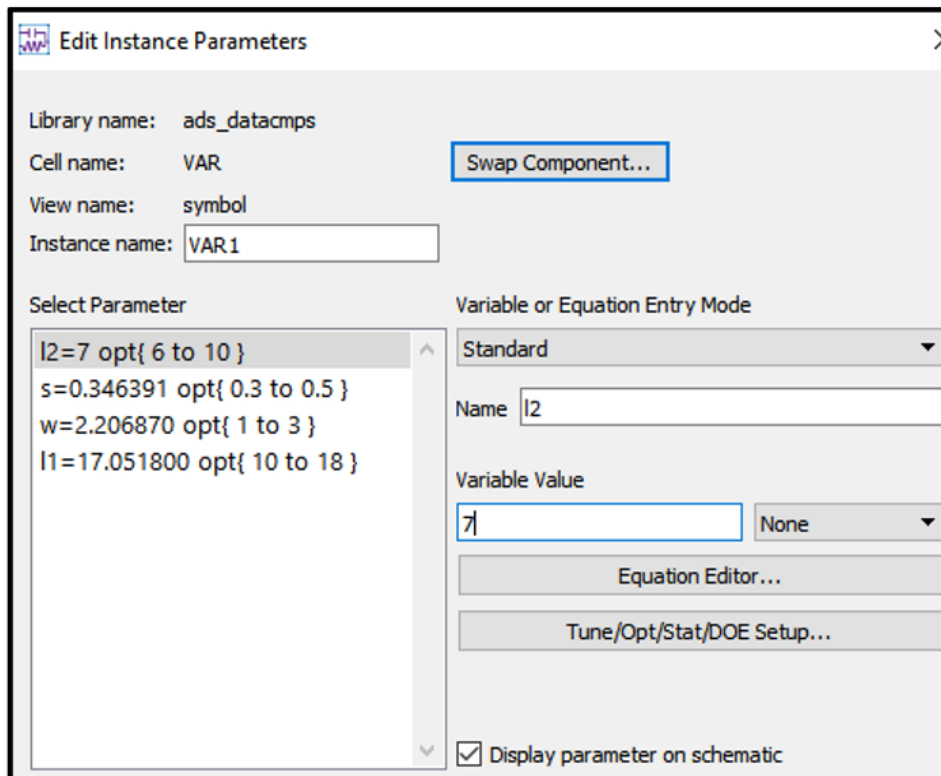
Qua 2 kết quả mô phỏng trên, ta thấy được các thông số khi sử dụng LineCalc chưa được tối ưu, khi vẫn còn các thông số chưa đạt kì vọng. Do đó, ta sử dụng 1 công cụ khác

của ADS để đạt được các thông số kì vọng mà ta đặt ra, đó là công cụ Optimization:



Hình 3.19: Schematic với các công cụ Optimization.

Đầu tiên, ta sử dụng khối VAR để tạo ra các biến mà ADS được phép thay đổi trong quá trình quá trình tối ưu. Các biến này được khai báo bao gồm giá trị khởi tạo và giới hạn trên/dưới, và sẽ đặt giá trị ban đầu theo thông số tính toán của LineCalc:



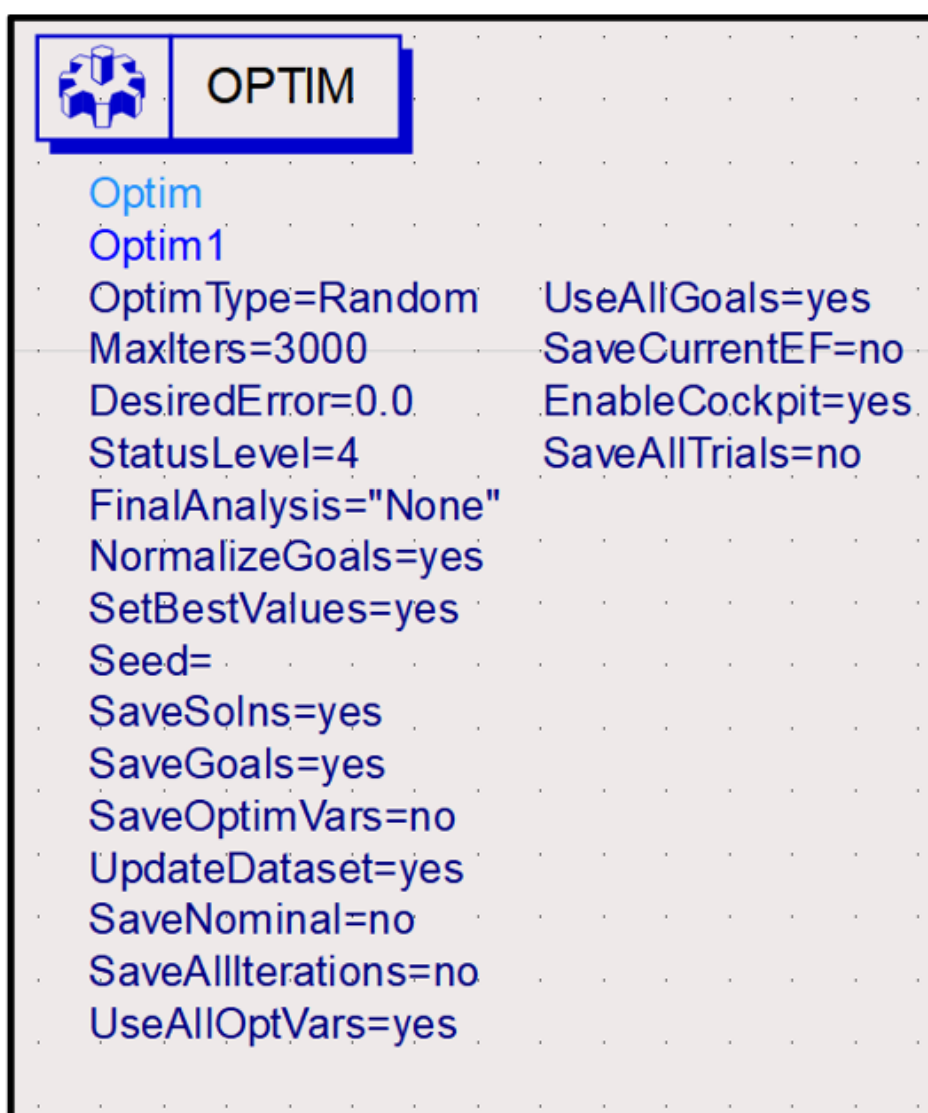
Hình 3.20: Khối VAR tạo các biến.

Tiếp theo là khối GOAL để đặt các mục tiêu mà mạch cần đạt được sau khi tối ưu. Ta sẽ đặt các thông số kì vọng như các kết quả mô phỏng ở trên:



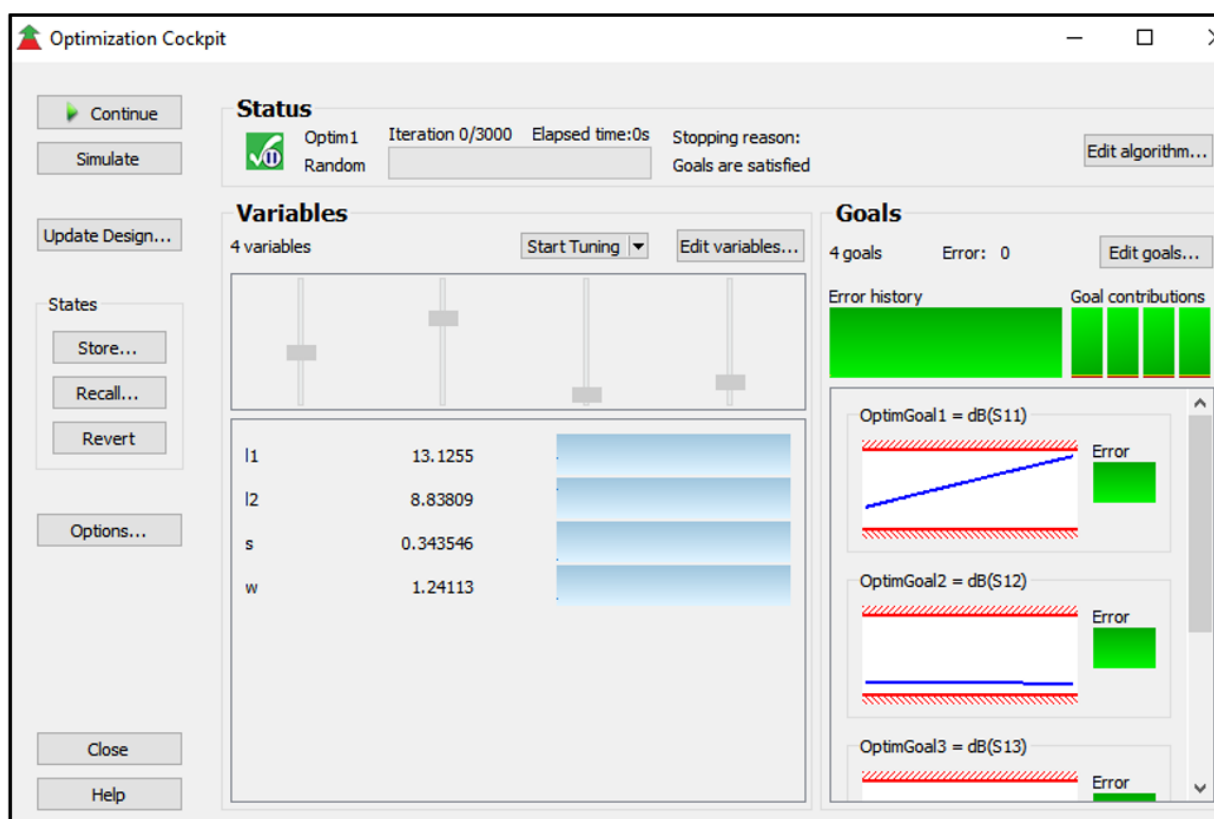
Hình 3.21: Khối GOAL đặt mục tiêu cần đạt được sau tối ưu.

Sau khi thiết lập biến và mục tiêu, khối OPTIM được thêm vào để lựa chọn thuật toán tối ưu phù hợp. Trong sơ đồ mô phỏng bên dưới, khối OPTIM được cấu hình để thực hiện quá trình tối ưu hóa bằng thuật toán Random Optimization. Tham số *MaxIters* = 3000 cho phép bộ tối ưu thực hiện tối đa 3000 vòng lặp nhằm tìm ra bộ giá trị tốt nhất cho các biến tối ưu.



Hình 3.22: Khối OPTIM để thực hiện quá trình tối ưu.

Sau khi thiết lập xong các công cụ Optimization, ta tiến hành thực hiện tối ưu hoá, tìm ra các thông số tính toán tối ưu nhất để đạt mục tiêu kì vọng:

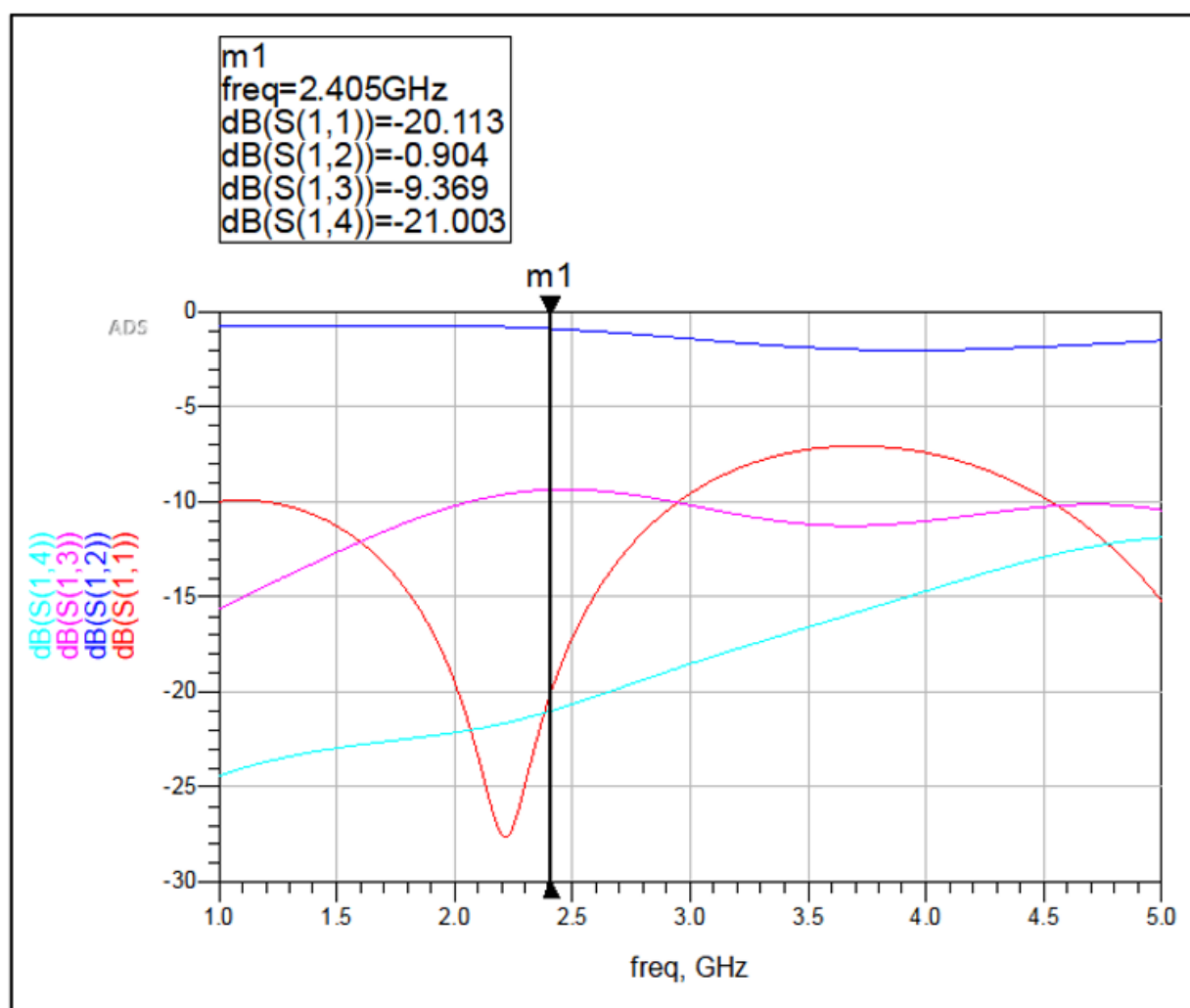


Hình 3.23: Kết quả sau khi tối ưu hóa.

Tiếp theo, ta tiến hành Update Design để cập nhật các thông số kích thước mới vào Schematic:

- $W = 1.24113 \text{ mm}$
- $S = 0.343546 \text{ mm}$
- $L_1 = 13.1255 \text{ mm}$
- $L_2 = 8.83809 \text{ mm}$

Kết quả mô phỏng Schematic sau khi tối ưu hoá:

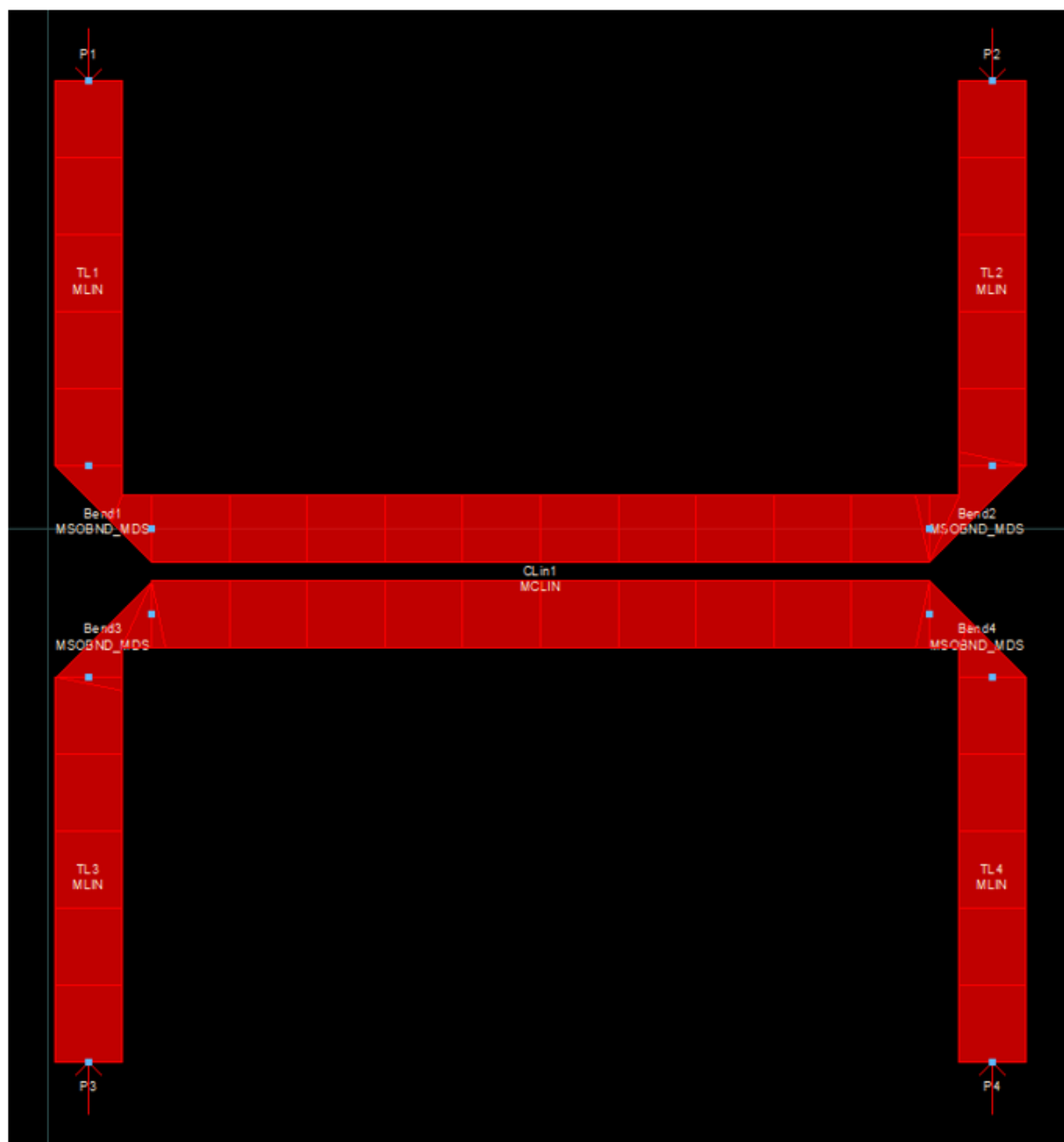


Hình 3.24: Kết quả mô phỏng Schematic sau khi tối ưu hóa.

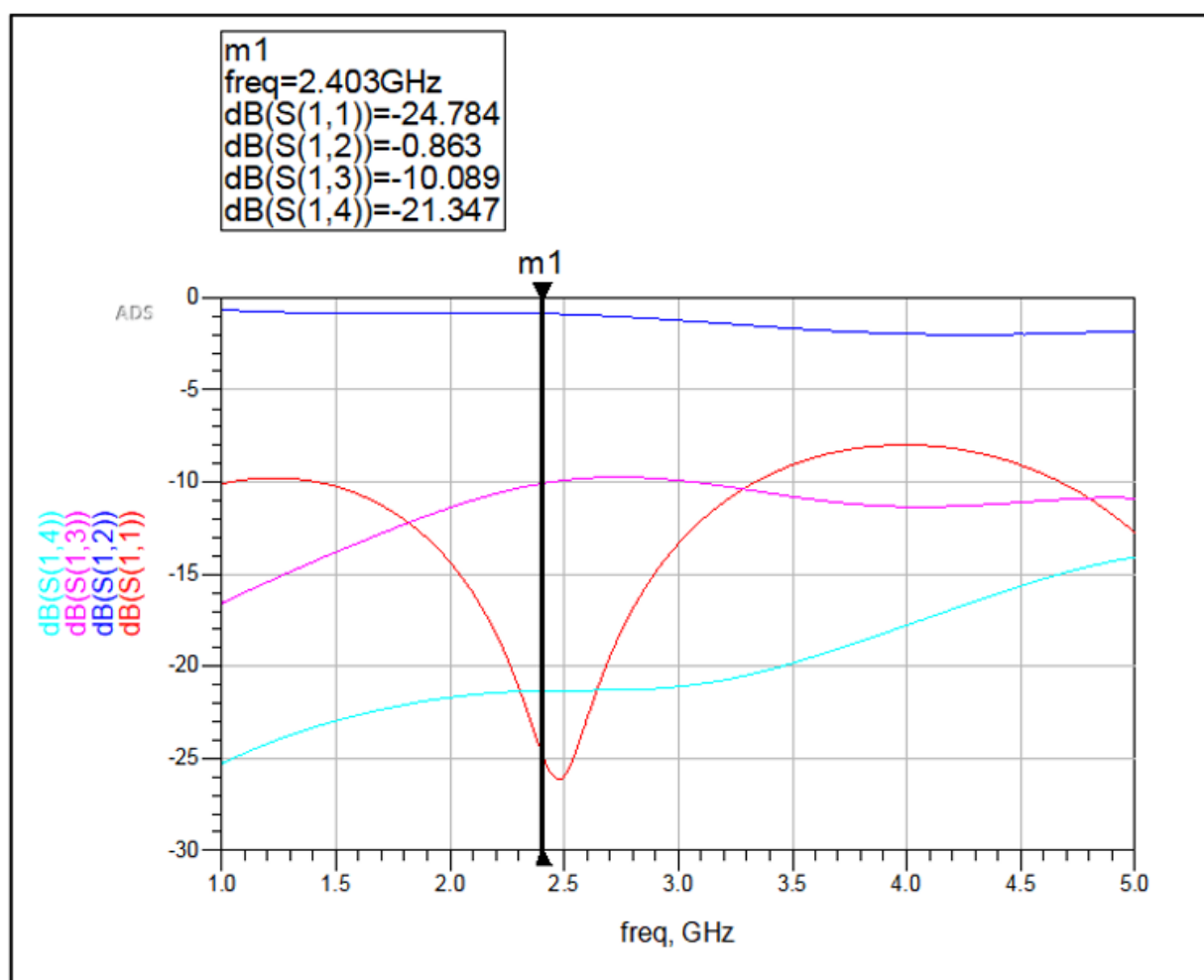
Đánh giá kết quả:

Thông số	Kì vọng	Đạt được	Đánh giá
S_{11}	Nhỏ hơn -20 dB	-20.113 dB	Đạt
S_{12}	Lớn hơn -1 dB	-0.904 dB	Đạt
S_{13}	Gần bằng -10 dB	-9.369 dB	Chấp nhận được
S_{14}	Nhỏ hơn -20 dB	-21.003 dB	Đạt

Bảng 3.3: Bảng so sánh thông số đo được và kì vọng khi mô phỏng Coupled line Directional Coupler sau khi Optimization.



Hình 3.25: Layout với các thông số kích thước tối ưu hoá.



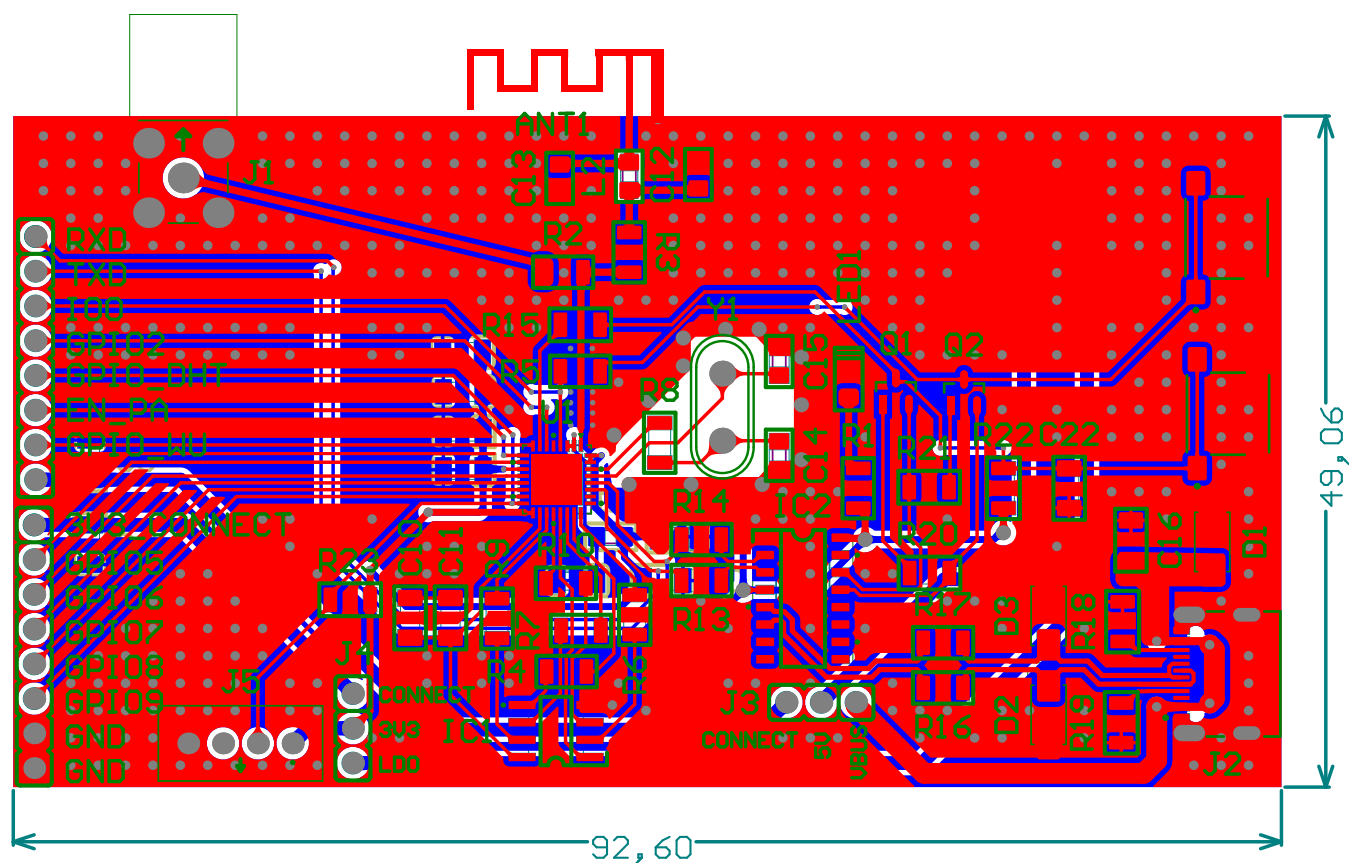
Hình 3.26: Kết quả mô phỏng Layout sau khi tối ưu hoá.

Thông số	Kì vọng	Đạt được	Đánh giá
S_{11}	Nhỏ hơn -20 dB	-24.784 dB	Đạt
S_{12}	Lớn hơn -1 dB	-0.863 dB	Đạt
S_{13}	Gần bằng -10 dB	-10.089 dB	Đạt
S_{14}	Nhỏ hơn -20 dB	-21.347 dB	Đạt

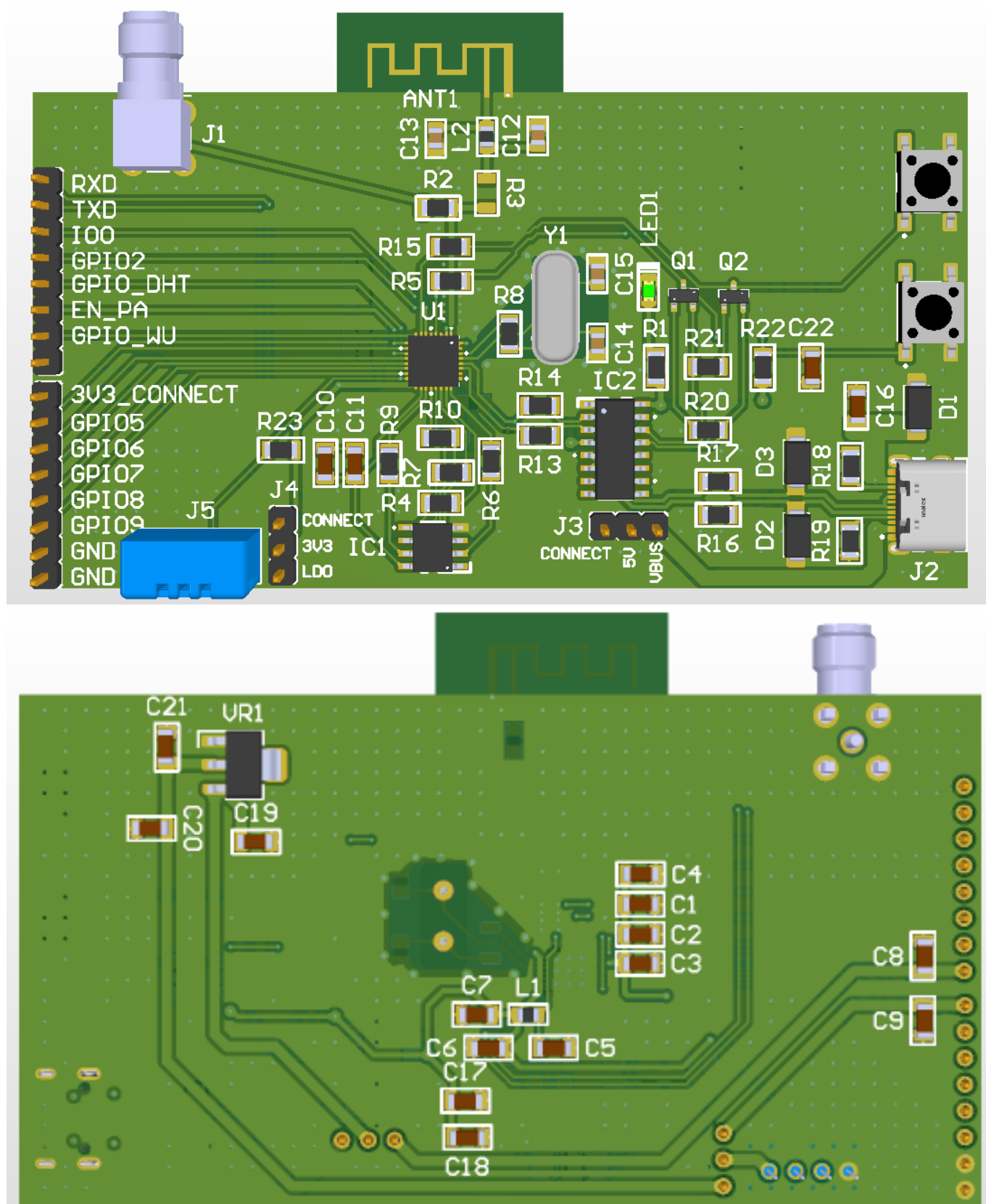
Bảng 3.4: Bảng so sánh thông số đo được và kì vọng sau khi chạy mô phỏng layout của Coupled Line Directional Coupler sau khi Optimization.

Chapter 4. Kết quả đo đặc và phân tích

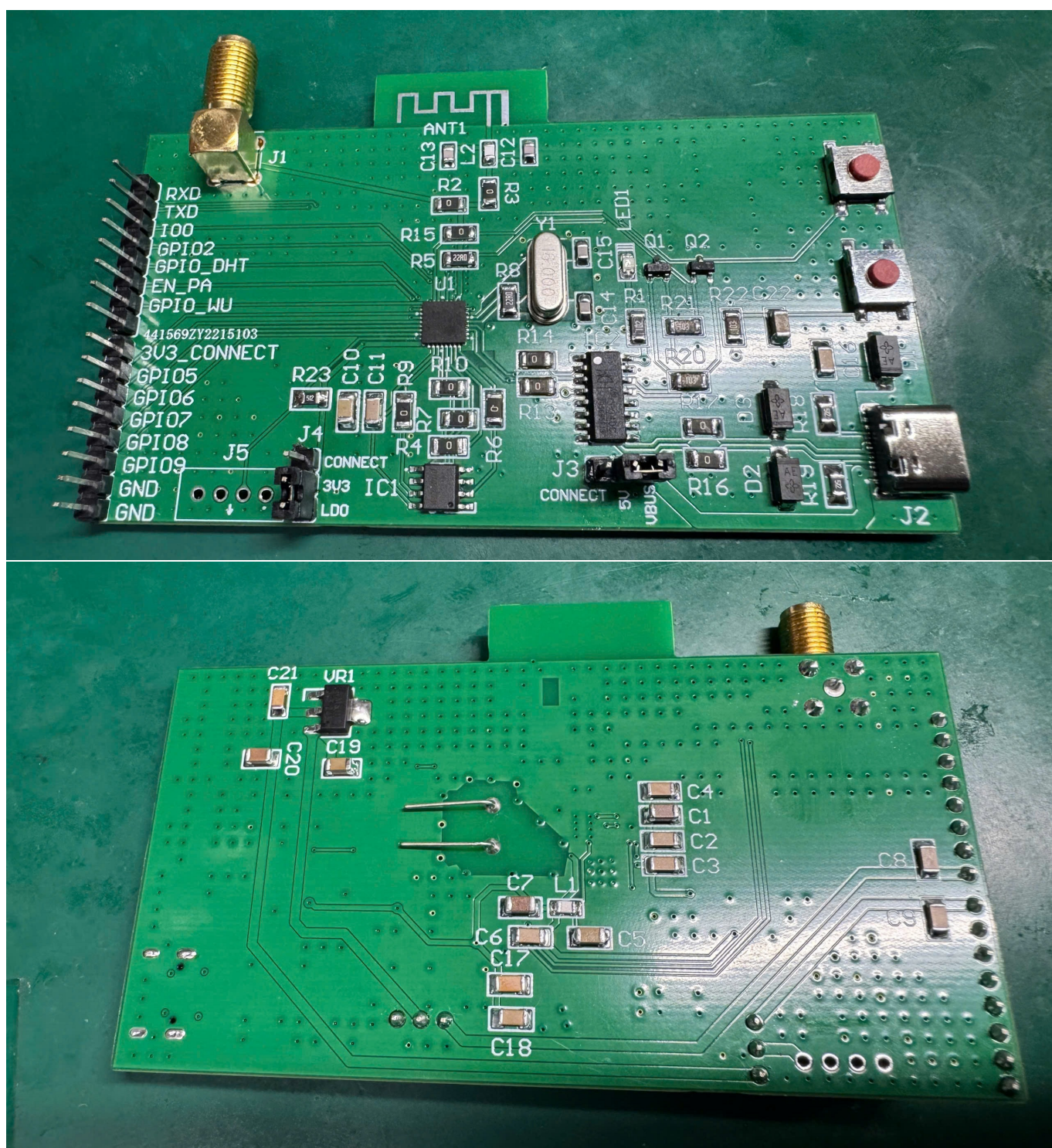
4.1 Kết quả của layout của một Sensor Node BLE



Hình 4.1: PCB tổng quát của mạch.

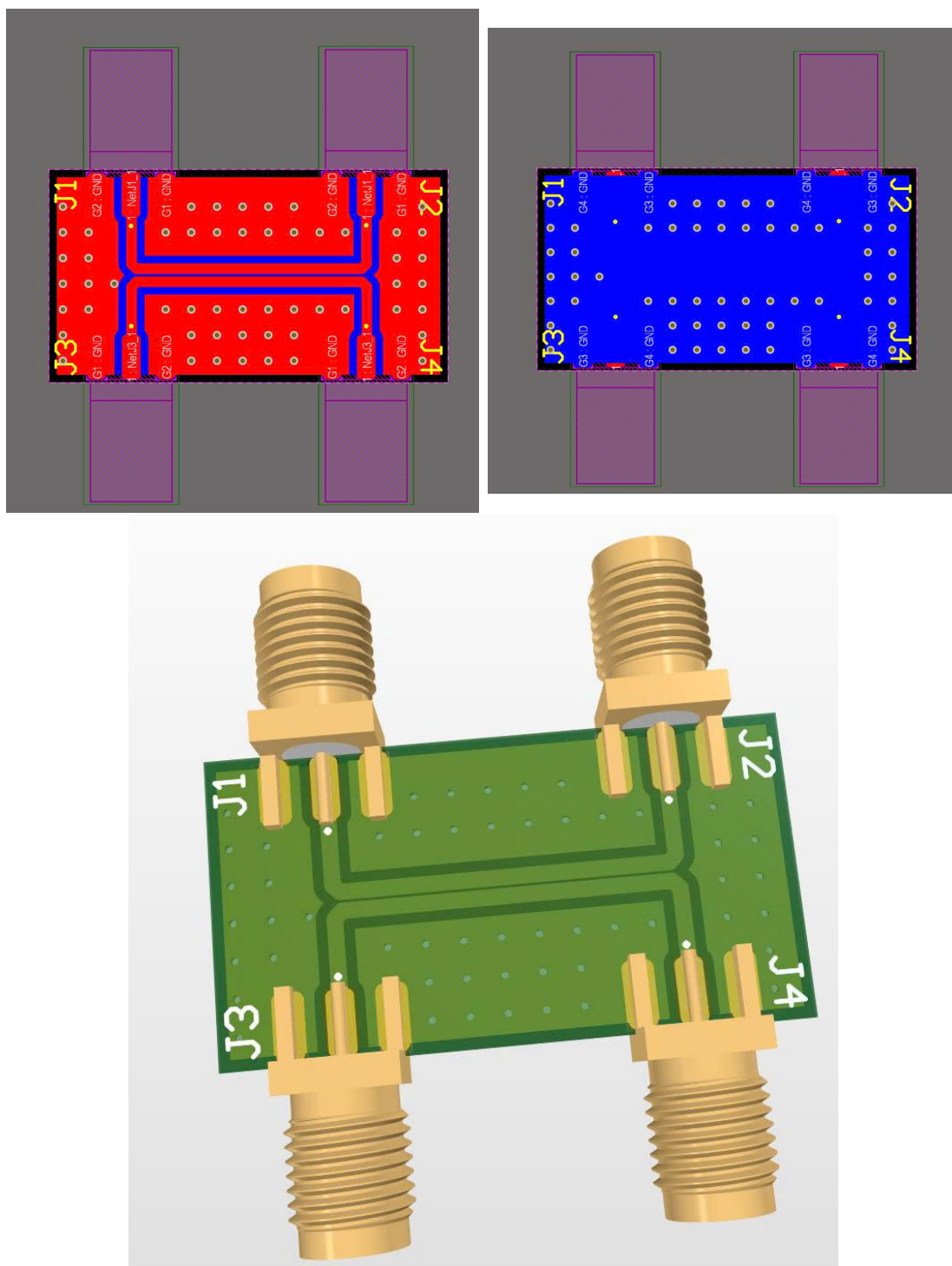


Hình 4.2: Hình ảnh 3D của mạch của một Sensor Node BLE.



Hình 4.3: Mạch thực tế của Sensor Node BLE.

4.2 Kết quả của layout của Directional Coupler



Hình 4.4: Hình ảnh 3D của mạch của Directional Coupler.

Tài liệu

- [1] M. Afaneh, “Bluetooth low energy (ble): A complete guide,” September 2022. Accessed: Oct. 6, 2025.
- [2] E. Reyes, M. G. Legaspi, and E. Peña, “Understanding the architecture of the bluetooth low energy stack,” December 2024. Accessed: Oct. 6, 2025.
- [3] R. Mischianti, “Esp32-c3: pinout, specs and arduino ide configuration,” April 2024. Accessed: Oct. 6, 2025.
- [4] P. Kindt *et al.*, “Precise energy modeling for the bluetooth low energy protocol,” *IEEE Internet of Things Journal*, vol. 6, no. 2, pp. 4005–4013, 2019.
- [5] M. Siekkinen *et al.*, “How low energy is bluetooth low energy? comparative measurements with zigbee/802.15.4,” *IEEE Wireless Communications*, vol. 20, no. 6, pp. 39–45, 2013.
- [6] N. Abdeddaim *et al.*, “Energy optimization in ble advertising for iot applications,” *Sensors*, vol. 20, no. 7, 2020.
- [7] S. Roundy *et al.*, “Energy scavenging for wireless sensor networks,” *Computer Communications*, vol. 26, pp. 1131–1144, 2003.
- [8] H. Malik *et al.*, “Performance comparison of ble and zigbee for iot,” *IEEE Access*, vol. 8, pp. 1139–1155, 2020.
- [9] A. Gomez *et al.*, “Ble mesh networking: Performance analysis and power consumption,” *IEEE Communications Magazine*, vol. 57, no. 5, pp. 107–113, 2019.
- [10] I. F. Akyildiz *et al.*, “A survey on sensor networks,” *IEEE Communications Magazine*, vol. 40, no. 8, pp. 102–114, 2002.
- [11] J. H. Hauer *et al.*, “Energy consumption modeling of ble devices for battery lifetime estimation,” *IEEE Sensors Journal*, vol. 21, no. 15, pp. 17123–17131, 2021.

PHỤ LỤC

Phụ lục A: Mã nguồn

```

1  #include <Arduino.h>
2  #include <BLEDevice.h>
3  #include <BLEServer.h>
4  #include <BLEUtils.h>
5  #include <BLE2902.h>
6  #include <DHT.h>
7  #include "esp_sleep.h"
8  #include "driver/rtc_io.h" // RTC pin library
9
10 // ----- TIME CONFIGURATION -----
11 // (Extended to give you comfortable connection time)
12 #define CONNECTION_TIMEOUT 60000 // 60 seconds waiting for connection
13 #define LIVE_DURATION 60000 // 60 seconds active time AFTER connection
14 #define SENSOR_READ_INTERVAL 5000 // Send sensor data every 5 seconds
15
16 // ----- DHT11 Configuration -----
17 // (Keep the same according to your wiring)
18 #define DHTPIN 4
19 #define DHTTYPE DHT11
20 DHT dht(DHTPIN, DHTTYPE);
21
22 // ----- GPIO Configuration -----
23 // (Keep the same according to your wiring)
24 #define LED_PIN 2 // On-board blue LED (P2)
25 #define BUTTON_PIN GPIO_NUM_33 // Wake-up button (P33)
26
27 // ----- BLE Configuration -----
28 #define SERVICE_UUID "4fafc201-1fb5-459e-8fcc-c5c9c331914b"
29 #define CHARACTERISTIC_UUID_TX "beb5483e-36e1-4688-b7f5-ea07361b26a9"
30
31 BLECharacteristic *pCharacteristicTX;
32 bool deviceConnected = false;
33
34 // ----- BLE Server Callback -----
35 // (Unchanged, this code is correct)
36 class MyServerCallbacks : public BLEServerCallbacks {
37     void onConnect(BLEServer *pServer) override {
38         deviceConnected = true;
39         digitalWrite(LED_PIN, HIGH);
40         Serial.println("[BLE] Device connected");
41     }
42
43     void onDisconnect(BLEServer *pServer) override {
44         deviceConnected = false;
45         digitalWrite(LED_PIN, LOW);
46         Serial.println("[BLE] Device disconnected");
47         pServer->getAdvertising()->start();
48     }
49 };
50
51 // --- SUPPORT FUNCTION: Read and Send Sensor Data ---
52 // (Unchanged, this code is correct)
53 void readAndSendSensorData() {
54     if (!deviceConnected) return;
55
56     float temperature = dht.readTemperature();
57     float humidity = dht.readHumidity();
58
59     if (isnan(temperature) || isnan(humidity)) {

```

```

60     Serial.println("[ERROR] Failed to read DHT sensor!");
61     pCharacteristicTX->setValue("Sensor Error!");
62 } else {
63     char sensorData[60];
64     sprintf(sensorData, "Temperature: %.1f*C, Humidity: %.1f%%", temperature, humidity);
65
66     Serial.print("[BLE] Sent: ");
67     Serial.println(sensorData);
68
69     pCharacteristicTX->setValue(sensorData);
70     pCharacteristicTX->notify();
71 }
72 }
73
74 // ----- Setup -----
75 // (Unchanged, this code is correct)
76 void setup() {
77     Serial.begin(115200);
78     pinMode(LED_PIN, OUTPUT);
79     delay(1000);
80
81     esp_sleep_wakeup_cause_t wakeup_reason = esp_sleep_get_wakeup_cause();
82     if (wakeup_reason == ESP_SLEEP_WAKEUP_EXT0) {
83         Serial.println("[SYSTEM] Woke up by button (GPIO33)");
84     } else {
85         Serial.println("[SYSTEM] Normal startup");
86     }
87
88     dht.begin();
89     BLEDevice::init("ESP32_DHT11_Node");
90     BLEServer *pServer = BLEDevice::createServer();
91     pServer->setCallbacks(new MyServerCallbacks());
92     BLEService *pService = pServer->createService(SERVICE_UUID);
93
94     pCharacteristicTX = pService->createCharacteristic(
95         CHARACTERISTIC_UUID_TX,
96         BLECharacteristic::PROPERTY_NOTIFY
97     );
98     pCharacteristicTX->addDescriptor(new BLE2902());
99     pService->start();
100     BLEDevice::getAdvertising()->start();
101     Serial.println("[BLE] Advertising started...");
102 }
103
104 // ----- Loop (FIXED VERSION) -----
105 void loop() {
106
107     // --- STEP 1: WAIT FOR CONNECTION ---
108     Serial.print("[SYSTEM] Waiting for BLE connection (Timeout: ");
109     Serial.print(CONNECTION_TIMEOUT / 1000);
110     Serial.println("s)");
111
112     unsigned long waitStartTime = millis();
113
114     // ----- FIX FOR ISSUE #147 -----
115     // Variables used for LED blinking (non-blocking)
116     unsigned long lastBlinkTime = 0;
117     bool ledState = false;
118
119     // Loop runs continuously without delay(250)

```

```

120 while (!deviceConnected && (millis() - waitStartTime < CONNECTION_TIMEOUT)) {
121
122     // Non-blocking LED blink logic
123     if (millis() - lastBlinkTime > 250) {
124         lastBlinkTime = millis();
125         ledState = !ledState;
126         digitalWrite(LED_PIN, ledState);
127     }
128
129     // IMPORTANT:
130     // Give CPU time for background tasks (e.g., BLE)
131     delay(10);
132 }
133 // ----- END FIX -----
134
135 // --- STEP 2: CONNECTION HANDLING ---
136 if (deviceConnected) {
137     // --- CONNECTED, STAY ALIVE ---
138     Serial.print("[SYSTEM] Connected. Staying active for ");
139     Serial.print(LIVE_DURATION / 1000);
140     Serial.println("s...");
141     digitalWrite(LED_PIN, HIGH); // Keep LED ON
142
143     unsigned long liveStartTime = millis();
144
145     // FIX: delay first sensor reading by 5s after connection
146     unsigned long lastSensorReadTime = millis();
147
148     while (millis() - liveStartTime < LIVE_DURATION) {
149
150         if (!deviceConnected) {
151             Serial.println("[SYSTEM] Disconnected during active period.");
152             break;
153         }
154
155         if (millis() - lastSensorReadTime >= SENSOR_READ_INTERVAL) {
156             lastSensorReadTime = millis();
157             readAndSendSensorData();
158         }
159
160         delay(100); // Small delay to allow BLE stack processing
161     }
162
163     Serial.println("[SYSTEM] Active period finished.");
164 } else {
165     // --- TIMEOUT WITHOUT CONNECTION ---
166     Serial.println("[SYSTEM] Connection timeout.");
167 }
168
169 // --- STEP 3: ENTER DEEP SLEEP ---
170 Serial.println("[SYSTEM] Entering deep sleep...");
171
172 rtc_gpio_pullup_en(BUTTON_PIN);
173 rtc_gpio_pulldown_dis(BUTTON_PIN);
174 esp_sleep_enable_ext0_wakeup(BUTTON_PIN, 0);
175
176 BLEDevice::deinit(true);
177 esp_deep_sleep_start();
178 }
179

```

Listing 2: Code thực thi chính của BLE sensor node.